

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY9. AI Agent 구현 실습 2

목 차

1. Coding agent		3
2. Business agent		26
3. deployment		53

1. Coding agent

이 스크립트 왜 안 돌지?

승승장구물 데이터팀이 넘겨준 매출 스크립트 data/buggy_script.py 가 에러를 뱉습니다. 한글 인코딩 누락, 문자열을 숫자처럼 더하기, 숫자를 문자열로 비교 — 흔한 버그 3 종이 섞여 있죠. 이번 장에서는 **LLM 이 코드를 읽고 버그를 찾아 고치는** 코딩 에이전트를 만듭니다.

1. 흔한 버그가 섞인 스크립트

데이터팀에서 받은 스크립트를 그냥 실행해 봅니다.

```
python data/buggy_script.py
```

예상 출력(에러)

```
Traceback (most recent call last):
...
TypeError: unsupported operand type(s) for +=: 'int' and 'str'
```

코드를 들여다보면 흔한 실수가 보입니다.

```
# buggy_script.py (버그가 섞인 원본)
with open("data/sales_daily.csv") as f:      # 버그1: encoding 미지정 → 한글 깨짐/에러
    rows = list(csv.DictReader(f))

total = 0
for r in rows:
    total += r["sales"]                       # 버그2: 문자열을 정수처럼 더함 (TypeError)

best = rows[0]
for r in rows:
    if r["sales"] > best["sales"]:            # 버그3: 숫자를 '문자열'로 비교 → 최댓값 틀림
        best = r
```

세 가지 모두 파이썬을 막 배운 사람도, 숙련자도 흔히 저지르는 정형적 버그입니다.

2. 세 가지 버그의 정체

버그	증상	원인
인코딩 누락	한글 깨짐 또는 <code>UnicodeDecodeError</code>	<code>open()</code> 에 <code>encoding="utf-8-sig"</code> 안 줌
문자열 합산	<code>TypeError: int + str</code>	CSV 값은 항상 문자열 → <code>int()</code> 변환 필요
문자열 비교	최댓값이 엉뚱하게 나옴	<code>"9" > "100"</code> 이 참(사전식 비교)

특히 **버그 3**이 무섭습니다. 에러가 안 나고 조용히 틀린 답을 주거든요. `"9" > "100"` 은 문자열 비교라 첫 글자 `'9' > '1'` 로 참이 됩니다. 에러 없이 잘못된 최댓값이 나오니, 사람이 눈치채기 어렵죠.

```
# 문자열 비교의 함정
print("9" > "100")    # True (사전식: '9' > '1') — 숫자로는 9 < 100인데!
print(int("9") > int("100")) # False (올바른 숫자 비교)
```

3. 사람이 디버깅해도 되지만...

물론 사람이 고칠 수 있습니다. 하지만:

- 코드가 길면 버그 찾는 데 시간이 걸림
- 비슷한 스크립트가 수십 개면 반복 작업
- 정형적 버그(인코딩 · 타입 · 비교)는 패턴이 뻔함

이런 정형적 버그는 **LLM**이 빠르게 잡아냅니다. 학습 데이터에 수많은 파이썬 코드와 버그 수정 사례가 있으니까요. 사람은 더 어려운 문제에 집중하고, 뻔한 버그는 에이전트에게 맡기는 게 효율적이죠.

4. 코딩 에이전트 — 읽고, 고치고, 검증

이번 강에서 만들 코딩 에이전트는 이렇게 동작합니다.

`data/buggy_script.py` (버그 3개)를 네 단계로 처리합니다.

1. 읽기 — 소스코드를 문자열로 로드
2. 수정 — LLM이 버그 찾아 고친 전체 코드 생성
3. 저장 — 새 파일(`fixed_script.py`)에 저장 — 원본은 보존!
4. 검증 — 실제로 실행 → 종료코드 0이면 성공

핵심은 "고쳤다는 말만 믿지 않고 실제로 돌려 본다" 입니다. LLM이 만든 코드는 그럴듯해 보여도 틀릴 수 있으니, 진짜 실행해서 정상 동작(종료코드 0)을 확인하죠. 이게 코딩 에이전트의 자가 검증 (self-verification) 입니다.

5. 백엔드 관점 — 자동 코드리뷰 보조

코딩 에이전트는 실무에서 코드리뷰/리팩터 보조 도구로 직접 쓸 수 있습니다.

- CI에 붙여: PR마다 "버그 의심 지점 + 수정안"을 코멘트로
- 내부 CLI: "이 스크립트 고쳐줘" → 수정본 새 파일 생성
- 레거시 정리: 오래된 스크립트의 인코딩 · 타입 버그 일괄 점검

단, 중요한 원칙이 있습니다(03번에서 자세히). 자동 수정은 "제안"으로 두고 사람이 검토 후 머지합니다. 원본을 절대 덮어쓰지 않고 새 파일/브랜치에 산출하죠. AI가 고친 코드를 검증 없이 바로 배포하는 건 위험하니까요.

1. 코딩 에이전트의 핵심 루프 — 읽기→수정→실행→검증 (02번)
2. 안전과 책임 — 원본 보존 · 타임아웃 · 사람 검토 (03번)
3. 버그 읽고 수정 요청하기 (04번)
4. 저장하고 실행해 검증하기 (05번)
5. [신규] 재귀적 버그 수정 — 오류를 다시 피드백하는 루프 (06번)

특히 06번의 재귀적 버그 수정이 백미입니다. 한 번 고쳐서 안 되면, 그 오류 메시지를 다시 LLM에 먹여 고치는 self-debugging 루프죠.

핵심 정리

- buggy_script.py 에는 흔한 버그 3 종: 인코딩 누락·문자열 합산·문자열 비교.
- 특히 문자열 비교("9" > "100")는 에러 없이 조용히 틀린 답을 쥐 위험.
- 정형적 버그는 LLM 이 빠르게 잡아냄 → 코딩 에이전트로 자동화.
- 핵심은 "고쳤다는 말만 믿지 않고 실제로 실행해 검증"(자가 검증).
- 백엔드로는 코드리뷰 보조(CI 코멘트·내부 CLI) — 단, 사람 검토 후 머지.

코딩 에이전트의 핵심 루프

코딩 에이전트의 핵심은 생성으로 끝내지 않고 실행해서 검증하는 것입니다.

코드 읽기 → 수정 생성 → 실행 → 오류 확인의 루프를 돌고, 오류가 남으면 다시 수정합니다.

이번 장은 기본형으로 1 회 수정 후 실행 검증까지 구현하고, 06 번에서 재귀 루프로 확장합니다.

1. 4단계 루프

[1. 코드 읽기] → [2. 수정 생성] → [3. 실행] → [4. 오류 확인]

4단계에서 오류가 남으면 2단계(수정)로 되돌아가 반복하고, 정상(종료코드 0)이면 완료합니다.

단계	하는 일	구현
1. 읽기	대상 소스 로드	<code>path.read_text()</code>
2. 수정	버그 고친 전체 코드 생성	LLM (<code>temperature=0</code>)
3. 실행	별도 프로세스로 돌림	<code>subprocess.run</code>
4. 확인	종료코드로 성공 판정	<code>returncode == 0</code>

핵심은 3·4단계입니다. 많은 사람이 "LLM이 코드를 만들면 끝"이라 생각하지만, 진짜 가치는 만든 코드를 실제로 돌려 검증하는 데 있습니다.

2. 왜 "실행 검증"이 핵심인가

LLM이 만든 코드는 그럴듯해 보여도 틀릴 수 있습니다.

LLM: "버그를 모두 고쳤습니다!" (자신만만)
→ 그런데 실행하면 여전히 에러 (혹은 새 버그 추가)

자연어 답변은 "맞는지" 사람이 읽어 봐야 알지만, 코드는 다릅니다 — 실행하면 객관적으로 판정됩니다.

종료코드 0 → 정상 동작 (객관적 성공)

종료코드 ≠ 0 → 에러 발생 (객관적 실패) + 오류 메시지 확보

이 객관적 검증 가능성이 코딩 에이전트를 강력하게 만듭니다. "고쳤다는 주장"이 아니라 "실제로 돌아간다는 증거"를 얻는 거죠. 16강(환각)에서 배운 "LLM 출력을 믿지 마라"를 코드로 실천하는 셈입니다.

3. 코드 추출 — LLM은 설명을 덧붙인다

2단계에서 한 가지 함정이 있습니다. LLM에게 "코드 고쳐줘"라고 하면 코드 앞뒤에 설명을 붙입니다.

LLM 응답:

"네, 버그를 분석했습니다. 먼저 인코딩 문제가..."

```
```python
```

```
import csv
```

```
...
```

이렇게 수정하면 정상 동작합니다."

이 응답을 그대로 파일로 저장해 실행하면 **SyntaxError**입니다(설명 문장은 파이썬이 아니니까). 그래서 코드블록만 추출하는 단계가 필요합니다.

```
import re
```

```
def extract_code(text: str) -> str:
```

```
 """LLM 응답에서 ```python ...``` 코드블록만 추출. 없으면 원문 반환."""
```

```
 m = re.search(r"```(?:python)?\s*(.*?)```", text, re.DOTALL) # 첫 코드블록 매칭
```

```
 return (m.group(1) if m else text).strip()
```

`re.DOTALL` 은 `.` 이 줄바꿈까지 매칭하게 해, 여러 줄 코드블록을 통째로 잡습니다. ````python` 과

````` 사이의 코드만 깔끔히 뽑아내죠.

4. 격리 실행 — subprocess

3단계 실행은 별도 프로세스(subprocess) 로 합니다. 왜 `exec()` 로 직접 실행하지 않을까요?

[exec()로 직접 실행] 내 프로그램 안에서 LLM 코드가 돌
→ LLM 코드가 죽으면 내 프로그램도 죽음
→ 무한루프면 내 프로그램도 멈춤
→ 위험!

[subprocess로 격리] 별도 프로세스에서 LLM 코드가 돌
→ 죽어도 내 프로그램은 멀쩡 (종료코드만 받음)
→ 타임아웃으로 무한루프 차단 가능
→ 안전!

```
import subprocess, sys

proc = subprocess.run(
    [sys.executable, str(FIXED)],      # 현재 파이썬으로 수정 파일 실행
    cwd=str(ROOT),                    # 작업 디렉터리 지정(상대경로 대비)
    capture_output=True, text=True,   # 출력을 문자열로 캡처
    timeout=60,                       # 무한루프 방지 타임아웃 (필수!)
)
print(proc.returncode)                # 0이면 정상
```

`timeout=60` 이 안전장치입니다. LLM이 실수로 무한루프 코드를 만들어도 60초 후 강제 종료되죠. 격리 실행은 코딩 에이전트의 기본 안전 원칙입니다(03번).

5. 기본형 → 재귀 루프 확장

이번 강은 **1회** 수정까지가 기본형입니다.

[기본형 (이번 강 04·05)] 읽기 → 수정 1번 → 실행 → 결과 보고
(실패하면 사람이 다시 돌림)

[재귀 루프 (06번 신규)] 읽기 → 수정 → 실행 → 실패면 오류를 피드백 → 재수정 → ...
(최대 N회 자동 반복, self-debugging)

1회 수정으로 대부분의 정형적 버그는 잡힙니다. 하지만 한 번에 안 되는 복잡한 버그를 위해, 06번에서 오류 메시지를 다시 LLM에 먹여 자동 재수정하는 루프로 발전시킵니다. 먼저 기본형부터 탄탄히 만들죠.

핵심 정리

- 코딩 에이전트 = 읽기 → 수정 → 실행 → 오류 확인 루프.
- 핵심은 실행 검증: 코드는 돌려 보면 종료코드로 객관적 판정 가능.
- LLM 은 코드에 설명을 덧붙임 → 정규식으로 코드블록만 추출(re.DOTALL).
- 실행은 subprocess 로 격리 + timeout 으로 무한루프 차단.
- 이번 강은 1 회 수정(기본형), 06 번에서 재귀 루프로 확장.

안전과 책임

코딩 에이전트는 코드를 만들고 실행하므로, 다른 어떤 에이전트보다 안전이 중요합니다. 원본 보존(새 파일에 저장), 타임아웃(무한루프 차단), 신뢰된 코드만 실행, 그리고 자동 수정은 제안일 뿐 사람이 최종 검토한다는 원칙을 지켜야 합니다.

1. 코드를 실행한다는 위험

코딩 에이전트는 LLM이 만든 코드를 실제로 돌립니다. 이건 강력하지만 동시에 위험합니다.

LLM이 만든 코드가...

- 무한루프라면? → 프로그램이 멈춤
- 파일을 지우는 코드라면? → 데이터 손실
- 원본을 덮어쓰는다면? → 되돌릴 수 없음
- 끝없이 메모리를 먹는다면? → 시스템 다운

자연어 챗봇은 "이상한 말"을 해도 사람이 안 따르면 그만이지만, 코드는 실행되면 실제로 일어납니다. 그래서 안전장치가 필수죠.

2. 안전장치 3종 + 1원칙

| 위험 | 대응 |
|-------------|---|
| LLM 코드의 부작용 | 원본 보존 — 새 파일에 저장, 원본은 비교 기준으로 그대로 |
| 무한 실행/위험 명령 | 타임아웃 — <code>timeout=60</code> , 신뢰된 코드만 실행 |
| 사람 검토 생략 | 제안 형태 — 자동 수정은 제안, 사람이 검토 후 머지 |

2-1. 원본 보존 — 새 파일에 저장

가장 중요한 원칙입니다. 원본을 절대 덮어쓰지 않습니다.

```
TARGET = DATA / "buggy_script.py"    # 고칠 대상 (원본, 절대 건드리지 않음)
FIXED = DATA / "fixed_script.py"      # 수정 결과는 '새 파일'에 저장

# 수정 코드는 FIXED에 쓴다 — TARGET은 그대로
FIXED.write_text(fixed_code, encoding="utf-8")
```

왜 원본을 보존할까요?

- 잘못 고쳤을 때 되돌릴 수 있음 (원본이 살아 있으니)
- 원본 vs 수정본을 비교(diff)할 수 있음
- "AI가 뭘 바꿨는지" 사람이 검토 가능

원본을 덮어썼는데 수정이 잘못됐다면? 복구 불가능입니다. 새 파일 원칙은 "되돌릴 수 있음"을 보장하는 안전망이죠.

2-2. 타임아웃 — 무한루프 차단

```
proc = subprocess.run([sys.executable, str(FIXED)],
                      timeout=60)          # 60초 넘으면 강제 종료
```

LLM이 실수로 `while True:` 같은 무한루프를 만들어도, 60초 후 자동 종료됩니다. 타임아웃 없이 실행하면 무한루프 코드 하나가 전체를 멈춥니다.

2-3. 신뢰된 코드만 실행

이번 강은 "우리가 만든 버그 스크립트"를 고치는 거라 안전하지만, 실무에서는 출처가 불분명한 코드를 함부로 실행하면 안 됩니다.

- 우리 레포의 스크립트 → 실행 OK
- 사용자가 던진 임의 코드 → 샌드박스/컨테이너에서 (10강 보안 참고)

3. 가장 중요한 원칙 — 사람이 최종 책임

기술적 안전장치보다 더 중요한 건 책임의 위치입니다.

[나쁜 흐름] 에이전트가 고침 → 검토 없이 바로 배포 → 사고
[좋은 흐름] 에이전트가 고침 → 사람이 검토 → 승인 → 머지/배포

자동 수정은 "제안(suggestion)"입니다. 최종 책임은 사람에게 있죠. AI가 고친 코드를 검증 없이 프로덕션에 올리는 건, 신입에게 코드리뷰 없이 main 브랜치 푸시 권한을 주는 것과 같습니다.

백엔드 관점: 코딩 에이전트를 CI 보조로 붙이면, PR에 대해 "버그 의심 지점 + 수정안"을 코멘트로 달게 할 수 있습니다. 단, 자동 수정 결과를 바로 배포하지 말고 **사람 리뷰**를 거치는 제안 형태로 두세요. 원본을 절대 덮어쓰지 말고 항상 **새 파일/브랜치**에 산출하세요.

4. 실무 적용 패턴

안전 원칙을 지키면서 코딩 에이전트를 쓰는 현실적 방법들입니다.

[CI 봇] PR 열리면 → 에이전트가 수정안 생성 → PR 코멘트로 제안
→ 개발자가 보고 채택 여부 결정

[로컬 CLI] "fix this script" → 새 파일로 수정본 생성
→ 개발자가 diff 확인 후 적용

[배치 점검] 레거시 스크립트 일괄 스캔 → 버그 의심 목록 리포트
→ 사람이 우선순위 정해 처리

공통점: 에이전트는 제안하고, 사람이 결정합니다. 이 구조를 지키면 코딩 에이전트는 강력한 보조 도구가 되죠.

5. 책임 있는 자동화의 가치

"왜 이렇게 조심하나, 그냥 자동으로 다 하면 안 되나?" 싶을 수 있습니다. 하지만:

빠르지만 위험한 자동화 → 한 번의 사고로 신뢰 붕괴
느려도 안전한 자동화 → 꾸준히 가치를 쌓음

코딩 에이전트의 목표는 **사람을 대체**하는 게 아니라 **사람을 돕는** 것입니다.

빠른 버그는 빠르게 잡아 주고, 최종 판단은 사람이 — 이 협업 구조가 책임 있는 AI 활용의 핵심이죠. 29 강(배포)에서 이 운영 원칙을 더 다룹니다.

핵심 정리

- 코딩 에이전트는 **코드를 실행**하므로 안전이 특히 중요.
- **원본 보존**: 새 파일에 저장 → 되돌리기·비교·검토 가능.
- **타임아웃**: timeout=60 으로 무한루프 차단.
- **신뢰된 코드만 실행**(임의 코드는 샌드박스).
- 가장 중요: 자동 수정은 "**제안**", 최종 책임은 **사람** — 검토 후 머지.
- 실무는 **CI 코멘트·로컬 CLI·배치 점검**으로 "제안하고 사람이 결정".

따라하기 — 버그 읽고 수정 요청

목표

버그가 있는 `data/buggy_script.py` 를 읽어 LLM에게 수정을 요청하고, 응답에서 **순수 코드만** 추출합니다. (실행 검증은 05번에서 붙입니다.)

0. 실습 준비

```
conda activate agentic
python code/common.py          # 환경/키 점검
python data/buggy_script.py     # 먼저 버그를 눈으로 확인 (에러 발생)
```

- 추가 설치 라이브러리 **없음** — `subprocess`, `re` 는 파이썬 표준 라이브러리, `get_chat` 은 앞 강에서 쓰던 헬퍼입니다.
- 대상 파일: `data/buggy_script.py` (버그 3개), 입력: `data/sales_daily.csv` (date, category, sales).

1. 임포트와 경로 설정

`code/ch23_coding.py`를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch23_coding.py
import sys, pathlib, subprocess, re
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, DATA

ROOT = pathlib.Path(__file__).resolve().parent.parent
TARGET = DATA / "buggy_script.py"          # 고칠 대상 (원본, 건드리지 않음)
FIXED = DATA / "fixed_script.py"           # 수정 결과를 저장할 새 파일
# [왜 원본 보존] 코딩 에이전트는 결과를 '새 파일'에 쓴다. 원본을 덮어쓰면
# 잘못 고쳤을 때 되돌릴 수 없으므로, 원본은 비교 기준으로 그대로 둔다.
```

`TARGET` (원본)과 `FIXED` (수정본)를 분리한 게 핵심입니다(03번 원본 보존). 수정은 항상 새 파일에 씁니다.

2. 대상 코드 읽기

```
def read_code(path: pathlib.Path) -> str:
    """대상 소스코드를 읽어 문자열로 반환."""
    return path.read_text(encoding="utf-8")
```

간단합니다. 파일을 통째로 문자열로 읽어 LLM에게 넘길 준비를 합니다. `encoding="utf-8"`로 한글 주석도 안전하게 읽죠.

3. LLM에게 수정 요청

핵심 함수입니다. "버그 고친 전체 코드만 코드블록으로 내놓아라" 라고 출력 형식을 못박습니다.

```
def ask_fix(source: str) -> str:
    """LLM에게 버그 수정된 전체 코드를 받아 순수 코드 문자열로 반환한다."""
    llm = get_chat(temperature=0) # [왜 0] 코드 수정은 일관 · 정확이 생명이라 무작위성 제거
```

```

prompt = (
    "다음 파이썬 스크립트에는 버그가 있다. 버그를 모두 찾아 고친 "
    "**전체 코드**를 하나의 코드블록으로만 출력하라. 설명·주석 외 텍스트는 쓰지 마라.\n"
    "주의: 한글 CSV 인코딩, 문자열/정수 타입 변환, 숫자 비교를 점검하라.\n\n"
    f"```python\n{source}\n```"      # 원본 코드를 코드블록에 담아 전달
)
out = llm.invoke(prompt).content
return extract_code(out)           # 응답에서 코드블록만 추출해 반환

```

프롬프트 설계 포인트

- `temperature=0` — 코드 수정은 **일관·정확**이 생명. 무작위성을 0으로(3·16강).
- **"전체 코드를 하나의 코드블록으로만"** — 일부만 고친 조각이 아니라 **실행 가능한 전체**를 요구.
- **"설명 텍스트는 쓰지 마라"** — 코드만 받아 저장하기 쉽게.
- **"인코딩·타입 변환·숫자 비교를 점검하라"** — 어디를 볼지 **힌트**를 줘 정확도를 높임.
- 원본 코드를 ````python ... ```` 블록에 담아 전달 → LLM이 "이게 입력 코드구나" 명확히 인식.

4. 코드블록만 추출

LLM이 설명을 덧붙여도(02번 함정) 코드블록만 깔끔히 뽑아냅니다.

```

def extract_code(text: str) -> str:
    """LLM 응답에서 ```python ... ``` 코드블록만 추출. 없으면 원문 반환.

    [왜 추출] LLM이 코드 앞뒤에 설명을 덧붙이는 경우가 많다. 그대로 실행하면
    SyntaxError 가 나므로, 정규식으로 코드블록 안쪽만 깔끔히 뽑아낸다.
    """
    m = re.search(r"```(?:python)?\s*(.*?)```", text, re.DOTALL) # 첫 코드블록 매칭
    return (m.group(1) if m else text).strip()

```

정규식 해부

| | |
|-----------------------------|--|
| <code>```(?:python)?</code> | → <code>```python</code> 또는 <code>```</code> (python은 있어도 없어도) |
| <code>\s*</code> | → 코드블록 시작 직후의 공백/줄바꿈 |
| <code>(.*?)</code> | → 코드 본문 (캡처 그룹 1) — ? 로 '최소 매칭'(첫 블록만) |
| <code>```</code> | → 코드블록 끝 |
| <code>re.DOTALL</code> | → . 이 줄바꿈까지 매칭 → 여러 줄 코드 통째로 |

코드블록이 없으면(`m`이 `None`) 원문을 그대로 반환합니다 — LLM이 코드만 깔끔히 줬을 수도 있으니까요.

5. 여기까지 확인

실행 검증(05번)을 붙이기 전에, 수정 코드가 잘 나오는지 출력해 봅니다.

```
if __name__ == "__main__":
    source = read_code(TARGET)
    print("[원본 일부]")
    print(source[:200], "...\\n")

    fixed = ask_fix(source)
    print("[LLM 수정본 일부]")
    print(fixed[:300], "...")
```

예상 출력

```
[원본 일부]
import csv
with open("data/sales_daily.csv") as f:
    rows = list(csv.DictReader(f))
total = 0
for r in rows:
    total += r["sales"] ...

[LLM 수정본 일부]
import csv
with open("data/sales_daily.csv", encoding="utf-8-sig") as f: # 인코딩 추가
    rows = list(csv.DictReader(f))
total = 0
for r in rows:
    total += int(r["sales"]) # 정수 변환 ...
```

관찰 포인트: LLM이 세 버그를 고친 흔적이 보입니다 — `encoding="utf-8-sig"` 추가, `int()` 변환. 하지만 아직 "정말 돌아가는지"는 모릅니다. "고친 것처럼 보인다"와 "진짜 고쳐졌다"는 다르죠. 그 검증을 다음 절에서 합니다.

핵심 정리

- TARGET(원본)과 FIXED(수정본)를 분리 → 원본 보존(03 번).
- `read_code()`로 소스를 문자열로 읽어 LLM 에 전달.
- `ask_fix()`: **temperature=0** + "전체 코드만 코드블록으로" + 점검 힌트.
- `extract_code()`: 정규식(`re.DOTALL`)으로 **코드블록만 추출** → `SyntaxError` 방지.

따라하기 — 저장하고 실행해 검증

목표

04번에서 받은 수정 코드를 새 파일로 저장하고 실제로 실행해, 종료코드로 정상 동작을 검증합니다. "고쳤다는 말"이 아니라 "진짜 돌아간다는 증거"를 얻습니다.

1. 저장하고 실행하는 함수

`code/ch23_coding.py`에 `save_and_run`을 추가합니다.

```
def save_and_run(code: str):
    """수정 코드를 새 파일로 저장한 뒤 별도 프로세스로 실행해 (종료코드, 출력)을 반환한다.

    [왜 실제 실행] '고쳤다'는 LLM 말만 믿지 않고, 진짜 돌려서 종료코드 0(성공)을
    확인한다 — 이것이 코딩 에이전트의 자체 검증(self-verification) 단계다.
    """
    FIXED.write_text(code, encoding="utf-8")      # 새 파일에 저장(원본 보존)
    # 원본이 'data/sales_daily.csv' 상대경로를 쓰므로 ROOT에서 실행
    proc = subprocess.run(
        [sys.executable, str(FIXED)],             # 현재 파이썬으로 수정 파일 실행
        cwd=str(ROOT), capture_output=True, text=True, timeout=60, # 무한루프 방지 타임아웃
    )
    return proc.returncode, (proc.stdout + proc.stderr) # 0=정상 종료, 출력은 합본
```

한 줄씩 짚기

- `FIXED.write_text(...)` — 수정 코드를 새 파일에 저장(원본 `TARGET`은 그대로).
- `[sys.executable, str(FIXED)]` — `sys.executable`은 지금 들고 있는 파이썬 경로. 같은 환경에서 실행.
- `cwd=str(ROOT)` — 작업 디렉터리를 프로젝트 루트로. 원본이 `data/sales_daily.csv` 상대경로를 쓰니, 루트에서 돌려야 파일을 찾습니다.
- `capture_output=True, text=True` — stdout/stderr를 문자열로 캡처.
- `timeout=60` — 무한루프 방지(03번 안전장치).
- 반환: (종료코드, 출력) — 종료코드 0이면 성공, 출력은 stdout+stderr 합본.

2. 왜 stdout과 stderr를 합치나

```
return proc.returncode, (proc.stdout + proc.stderr)
```

`stdout` (정상 출력)과 `stderr` (에러 메시지)를 합쳐서 반환합니다. 이유:

- 성공 시: stdout에 결과("평균 매출: ...") → 이걸 보여줌
- 실패 시: stderr에 에러(Traceback ...) → 이걸 06번에서 LLM에 피드백

둘 다 잡아 두면, 성공이든 실패든 무슨 일이 있었는지 한 변수로 확인할 수 있죠. 특히 실패 시의 에러 메시지는 06번 재귀 수정의 핵심 입력이 됩니다.

3. 전체 파이프라인 실행

읽기 → 수정 → 저장·실행을 잇습니다.

```
if __name__ == "__main__":
    print("[1] 원본 코드 읽기:", TARGET.name)
    source = read_code(TARGET)          # 버그 있는 원본 읽기

    print("[2] LLM에게 버그 수정 요청 중...")
    fixed = ask_fix(source)              # LLM이 고친 전체 코드 받기

    print("[3] 새 파일로 저장 후 실행 검증:", FIXED.name)
    code, output = save_and_run(fixed)   # 저장→실행→종료코드로 검증

    print("=" * 60)
    print("[실행 결과] 종료코드:", code, "(0=정상)")
    print(output)
    print("=" * 60)
    if code == 0:
        print("성공: 버그가 수정되어 정상 동작합니다 →", FIXED)
    else:
        print("실패: 여전히 오류가 있습니다. 출력을 LLM에 다시 피드백해 재수정하세요.")
```

예상 출력(성공)

```
[1] 원본 코드 읽기: buggy_script.py
[2] LLM에게 버그 수정 요청 중...
[3] 새 파일로 저장 후 실행 검증: fixed_script.py
```

[실행 결과] 종료코드: 0 (0=정상)

평균 매출: 152437.5

최대 매출일: 2024-01-01

성공: 버그가 수정되어 정상 동작합니다 → /Users/.../data/fixed_script.py

종료코드 **0**이 떴습니다. "고쳤다는 LLM 말"이 아니라 "실제로 돌아간다는 객관적 증거"를 얻은 거죠. 이게 자가 검증(self-verification)입니다.

4. 고쳐진 부분 확인

수정본(`data/fixed_script.py`)을 열어 보면 세 버그가 고쳐져 있습니다.

```
with open(path, encoding="utf-8-sig") as f: # 버그1: 인코딩 지정 (한글 BOM 대비)
...
total += int(r["sales"]) # 버그2: 문자열 → 정수 변환
...
if int(r["sales"]) > int(best["sales"]): # 버그3: 숫자로 비교 (문자열 비교 x)
```

원본(`buggy_script.py`)은 그대로 있습니다. diff를 떠 보면 "AI가 정확히 어디를 바꿨는지" 사람이 검토할 수 있죠(03번 원본 보존의 가치).

```
diff data/buggy_script.py data/fixed_script.py
```

5. 실패하면? — 06번 예고

만약 한 번에 안 고쳐지면(종료코드 $\neq 0$), 출력에 에러가 남습니다.

```
[실행 결과] 종료코드: 1 (0=정상)
Traceback (most recent call last):
  File ".../fixed_script.py", line 8, in <module>
    total += int(r["sales"])
ValueError: invalid literal for int() with base 10: '1,500' ← 천단위 콤마!
```

실패: 여전히 오류가 있습니다. 출력을 LLM에 다시 피드백해 재수정하세요.

이 경우 에러 메시지를 다시 LLM에 먹여 "이 오류가 사라지게 고쳐줘"라고 하면 됩니다. 이게 06번의 재귀적 버그 수정(**self-debugging** 루프)입니다. 지금은 "1회 수정 + 검증"까지 완성했고, 다음 절에서 자동 반복으로 확장합니다.

6. 모듈화 — 재사용 가능한 도구로

`read_code` / `ask_fix` / `save_and_run` 을 분리해 둔 덕에, 이것 함수 하나로 묶어 재사용할 수 있습니다.

```
def fix_file(path: pathlib.Path):
    """파일 하나를 고쳐 새 파일로 저장하고 (성공여부, 출력)을 반환하는 재사용 함수."""
    source = read_code(path)
    fixed = ask_fix(source)
    code, output = save_and_run(fixed)
    return code == 0, output

# CLI · CI 혹은 내부 도구에서 호출
ok, out = fix_file(DATA / "buggy_script.py")
print("성공" if ok else "실패", out[:100])
```

이렇게 묶으면 CLI 명령(`fix script.py`), CI 혹은 내부 도구로 깔끔히 불일 수 있죠. 실행은 반드시 타 임아웃을 뒤 무한루프로부터 보호하는 걸 잊지 마세요.

핵심 정리

- `save_and_run()`: 수정 코드를 새 파일 저장 → **subprocess** 실행 → 종료코드 반환.
- `cwd=ROOT`(상대경로 대비), `timeout=60`(무한루프 차단), `stdout+stderr` 합본.
- 종료코드 **0** = 객관적 성공 증거 — "고쳤다는 말"이 아닌 "진짜 돌아감".
- 원본은 보존되어 **diff** 로 변경 검토 가능.
- 실패 시 에러를 다시 피드백 → 06 번 재귀 수정으로 확장.
- 세 함수를 `fix_file()`로 묶어 **CLI·CI** 도구로 재사용.

재귀적 버그 수정 (오류 피드백 루프)

한 번 고쳐서 안 되면, 그 오류 메시지를 다시 LLM 에 먹여 재수정합니다.

수정→실행→실패면 오류 피드백→재수정을 최대 N 회 반복하는 self-debugging 루프입니다.

단, 무한 재시도를 막는 횟수 상한이 필수입니다.

1. 1회 수정의 한계

05번은 "수정 1번 → 실행 → 결과 보고"였습니다. 대부분의 정형적 버그는 한 번에 잡히지만, 가끔 안 됩니다.

1차 수정: `int()` 변환 추가

→ 실행: `ValueError: invalid literal for int() with base 10: '1,500'`

→ 천단위 콤마(1,500)는 `int()`가 못 읽음! 새 문제 발견

한 번에 안 보였던 문제(콤마)가 실행해 보니 드러난 거죠. 사람이라면 이 에러를 보고 "아, 콤마를 제거해야겠네" 하고 다시 고칩니다. 이 과정을 자동화하는 게 재귀적 버그 수정입니다.

2. 핵심 아이디어 — 오류를 피드백으로

사람의 디버깅을 그대로 코드로 옮깁니다.

- [사람의 디버깅] 고친다 → 돌려본다 → 에러 본다 → "아 이게 문제구나" → 다시 고친다 → ...
- [에이전트의 self-debugging] `ask_fix` → `run` → 실패면 에러를 `ask_fix` 에 피드백 → 재수정 → `run` → ... (이 '에러 피드백'이 핵심!)

LLM에게 직전 실행에서 난 실제 오류 메시지를 주면, 그 오류를 표적으로 정확히 고칩니다. "추측"이 아니라 "실제 에러 기반 수정"이라 정확도가 훨씬 높죠.

3. 오류를 받는 ask_fix

`ask_fix`에 `error` 인자를 추가해, 오류가 있으면 프롬프트에 포함합니다.

```
# 실행: python "sections/23_coding_agent/06_재귀적-버그수정-오류-피드백-루프.py"
import sys, pathlib, subprocess
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat, DATA
from ch23_coding import read_code, extract_code # 기존 도우미 재사용

ROOT = pathlib.Path(__file__).resolve().parents[2]
TARGET = DATA / "buggy_script.py" # 원본(절대 안 건드림)
FIXED = DATA / "fixed_script_loop.py" # 수정 결과 새 파일
MAX_TRIES = 3 # [경고] 무한 재시도 방지 — 최대 시도 횟수 고정

def ask_fix(source: str, error: str = "") -> str:
    """버그 수정 전체 코드를 LLM에서 받는다. error가 있으면 그 오류를 피드백에 포함."""
    llm = get_chat(temperature=0) # [왜 0] 코드 수정은 일관·정확이 생명
    feedback = ""
    if error:
        # [핵심] 직전 실행에서 난 실제 오류 메시지를 그대로 LLM에 전달 → 표적 수정.
        feedback = (
            "\n\n[직전 실행 오류 — 이 오류가 사라지도록 반드시 고쳐라]\n"
            f"{error[:1500]}\n"
        )
    prompt = (
        "다음 파이썬 스크립트에는 버그가 있다. 버그를 모두 찾아 고친 "
        "**전체 코드**를 하나의 코드블록으로만 출력하라. 설명 텍스트는 쓰지 마라.\n"
        "주의: 한글 CSV 인코딩, 문자열/정수 타입 변환, 숫자 비교를 점검하라."
        f"{feedback}\n"
        f"```python\n{source}\n```"
    )
    return extract_code(llm.invoke(prompt).content)
```

짚어 둘 점

- `error=""` 기본값 — 첫 시도엔 오류가 없으니 빈 문자열(05번과 동일하게 동작).
- `error[:1500]` — 에러 메시지가 길 수 있어 1500자로 자름(토큰 관리).
- 피드백 문구를 **명령형**으로 ("이 오류가 사라지도록 반드시 고쳐라") → LLM이 표적 수정.
- `from ch23_coding import read_code, extract_code` — 05번에서 만든 도우미를 **재사용**(중복 제거).

4. 실행 함수

05번 `save_and_run` 과 같지만, 별도 파일명(`fixed_script_loop.py`)을 씁니다.

```
def run_code(code: str):
    """수정 코드를 새 파일로 저장 후 별도 프로세스로 실행해 (종료코드, 출력) 반환."""
    FIXED.write_text(code, encoding="utf-8")
    proc = subprocess.run(
        [sys.executable, str(FIXED)],
        cwd=str(ROOT), capture_output=True, text=True, timeout=60, # 무한루프 방지
    )
    return proc.returncode, (proc.stdout + proc.stderr)
```

5. self-debugging 루프

핵심입니다. 최대 `MAX_TRIES` 회까지 수정→실행→실패면 피드백→재수정을 반복합니다.

```
if __name__ == "__main__":
    print("[0] 원본 코드 읽기:", TARGET.name)
    source = read_code(TARGET)
    error = "" # 첫 시도엔 피드백 없음
    success = False

    for i in range(1, MAX_TRIES + 1):
        print("=" * 60)
        print(f"[시도 {i}/{MAX_TRIES}] {'오류 피드백으로 재수정' if error else '최초 수정'}")
        fixed = ask_fix(source, error) # (재)수정 코드 받기
        code, output = run_code(fixed) # 저장→실행→검증
        print(f"종료코드: {code} (0=정상)")
        if code == 0:
            print("결과: 정상 동작")
            print(output.strip())
            success = True
            break # 성공 → 루프 탈출
        # 실패 → 오류를 다음 시도의 피드백으로 넘기고, 수정본을 다음 입력으로 사용
        print("결과: 여전히 오류 →")
        print(" " + output.strip().replace("\n", "\n "))
        error = output # [핵심] 이번 오류를 다음 시도에 피드백
        source = fixed # [핵심] 다음 라운드는 '직전 수정본'을 기준으로 더 고친다
```

```
print("=" * 60)
if success:
    print("성공: self-debugging 루프로 버그를 잡았습니다 →", FIXED)
else:
    print(f"실패: {MAX_TRIES}회 시도에도 해결되지 않았습니다(사람 검토 필요).")
```

루프의 두 가지 핵심

```
error = output    → 이번 실행의 오류를 다음 ask_fix에 피드백 (표적 수정)
source = fixed    → 다음 라운드는 '직전 수정본'을 기준으로 더 고침 (누적 개선)
```

매 라운드 직전 수정본 + 직전 오류를 입력으로 줘서, 점점 나아지게 합니다. 처음부터 다시 고치는 게 아니라 이전 시도 위에 쌓는 거죠.

6. 실행 결과

예상 출력(2회 만에 성공)

```
[0] 원본 코드 읽기: buggy_script.py
=====
[시도 1/3] 최초 수정
종료코드: 1 (0=정상)
결과: 여전히 오류 →
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: '1,500'
=====
[시도 2/3] 오류 피드백으로 재수정
종료코드: 0 (0=정상)
결과: 정상 동작
평균 매출: 152437.5
최대 매출일: 2024-01-01
=====
성공: self-debugging 루프로 버그를 잡았습니다 → /Users/.../data/fixed_script_loop.py
```

1차에서 콤마 문제로 실패했지만, 그 오류를 피드백받은 2차에서 콤마 제거(`replace(",", "")`)를 추가해 성공했습니다. 사람의 디버깅 과정을 그대로 자동화한 거죠.

7. 무한 재시도 방지 — 가장 중요한 안전장치

`MAX_TRIES = 3` 이 필수입니다. 왜일까요?

[상한 없는 루프] `while not success:` 재수정 ...
→ LLM이 못 고치는 버그면 → 무한 반복 → 비용 폭발 · 멈춤

[상한 있는 루프] `for i in range(MAX_TRIES):` ...
→ N회 시도 후 포기 → "사람 검토 필요" 보고

자동화에는 항상 탈출구가 필요합니다(7강 `max_steps` 정신). LLM이 3번 시도해도 못 고치는 버그는, 사람이 봐야 할 어려운 문제일 가능성이 높죠. 그럴 땐 깔끔히 포기하고 사람에게 넘기는 게 옳습니다. 무한 루프로 비용을 태우는 것보다요.

8. 비용 트레이드오프

재귀 루프는 강력하지만 공짜가 아닙니다.

1회 수정(05번): LLM 호출 1번 + 실행 1번
재귀 루프(06번): LLM 호출 최대 3번 + 실행 최대 3번 → 비용 · 시간 최대 3배

그래서 실무에서는 `MAX_TRIES`를 적절히(보통 2~3회) 잡습니다. 대부분의 버그는 1~2회면 잡히고, 그 이상 안 되면 사람이 봐야 할 문제니까요. "무한히 시도해서 반드시 고친다"가 아니라 "몇 번 해 보고 안 되면 넘긴다"가 현실적입니다.

핵심 정리

- 1회 수정으로 안 되면 오류 메시지를 다시 LLM에 피드백해 재수정.
- self-debugging 루프 = 수정→실행→실패면 오류 피드백→재수정 반복.
- 핵심 두 줄: `error = output(오류 피드백) + source = fixed(수정본 누적)`.
- `MAX_TRIES` 상한 필수 — 무한 재시도·비용 폭발 방지(7강 탈출구).
- 비용은 최대 N배 → 보통 2~3회로 제한, 안 되면 사람 검토로 넘김.

실습문제

문제 1 — 변경 요약 출력 추가

`code/ch23_coding.py`를 복사해 `code/ch23_ex_summary.py`를 만드세요. `ask_fix`의 프롬프트를 확장해 수정 코드와 함께 "어떤 버그를 왜 고쳤는지" 변경 요약을 받아오게 하되, 요약은 콘솔에만 출력하고 `data/fixed_script.py`에는 코드만 저장하도록 분리하세요.

- 사용 파일: `data/buggy_script.py`
- 기대 결과: 콘솔에 버그 3건(인코딩·문자열 합산·문자열 비교)에 대한 변경 요약이 출력되고, 저장 파일은 정상 실행된다.

문제 2 — 새 버그 스크립트 작성·수정

`data/my_buggy.py`를 직접 만들어 의도적 버그 2개(예: 0으로 나누기, 잘못된 인덱싱)를 넣으세요. `code/ch23_coding.py`의 `TARGET`을 `data/my_buggy.py`로 바꿔(또는 인자로 받게 수정) 실행하세요.

- 검증: 에이전트가 출력·저장한 코드를 직접 실행해 두 버그가 모두 사라지고 정상 종료(종료코드 0) 되는지 확인.

2. Business agent

숫자는 있는데 보고서가 없다

승승장구물 경영진은 매달 "지난달 매출 어땠어?"를 묻습니다. 데이터는 `monthly_sales.csv` 에 다 있지만, 표 그대로는 읽기 불편하죠. 누군가 매달 숫자를 뽑고 → 추세를 해석하고 → 문장으로 정리해야 합니다. 이번 강에서는 이 반복 작업을 자동화하는 **비즈니스 리포트 에이전트**를 만듭니다.

1. 표는 데이터지 보고서가 아니다

`monthly_sales.csv` 를 열면 이렇게 생겼습니다.

| month | 전자기기 | 가전 | 식품 | ... | total |
|---------|-----------|-----------|-----------|-----|------------|
| 2026-03 | 3,210,000 | 2,800,000 | 1,500,000 | | 9,760,000 |
| 2026-04 | 3,500,000 | 2,900,000 | 1,600,000 | | 9,760,000 |
| 2026-05 | 4,449,850 | 3,418,050 | 1,800,000 | | 12,404,300 |

데이터는 다 있습니다. 하지만 경영진이 원하는 건 표가 아니라 "그래서 어땠는데?" 입니다.

경영진: "5월 매출 어땠어?"
원하는 답: "5월 총매출 1,240만원으로 전월 대비 27% 증가했고,
전자기기가 성장을 견인했습니다. 다음 달은 ..."
원하지 않는 답: (엑셀 표를 그대로 들이밀기)

표를 읽을 수 있는 보고서로 바꾸는 일 — 이게 매달 반복되는 수작업입니다.

2. 매달 반복되는 보고서 작업

담당자가 매달 하는 일을 뜯어보면:

1. CSV에서 지난달 총매출 계산
2. 전월과 비교해 증감률 계산
3. 카테고리별로 정렬해 1·2위 파악
4. "왜 늘었나/줄었나" 해석
5. 경영진이 읽을 문장으로 정리
6. 다음 달 제언 추가
7. 문서로 저장·발송

1~3번은 **계산**(기계적), 4~6번은 **서술**(글쓰기), 7번은 **저장**(자동화 가능). 매달 똑같은 패턴이죠. 이 반복을 에이전트에게 맡깁니다.

3. 핵심 통찰 — LLM의 강점은 계산이 아니라 서술

여기서 중요한 설계 결정이 있습니다. "**계산을 누가 하느냐**" 입니다.

[나쁜 방법] LLM에게 CSV를 통째로 주고 "총매출 계산해서 보고서 써줘"

→ LLM이 숫자를 틀리거나 지어낼 수 있음 (산술 환각!)

→ "총매출 1,240만원"이 "1,420만원"으로 둔갑

[좋은 방법] 숫자는 코드(pandas)로 정확히 계산 → LLM은 그 숫자로 글만 씀

→ 숫자는 항상 정확, LLM은 잘하는 '서술'에만 집중

10강에서 배웠듯 **LLM은 계산기가 아닙니다**. 큰 숫자의 곱셈·증감률을 머릿속으로 하면 틀립니다. 반대로 요약·해석·문장화는 **LLM의 진짜 강점**이죠. 그래서 둘을 나눕니다 — **계산은 코드, 서술은 LLM**(02번에서 자세히).

4. 우리가 만들 것

이번 강의 비즈니스 에이전트는 이렇게 동작합니다.

`monthly_sales.csv` 를 세 단계로 처리합니다.

1. 집계 (**pandas**) — 총매출·증감률·카테고리 1·2위를 코드로 정확히 계산 → `facts` (확정 수치)

2. 서술 (**LLM**) — `facts` 를 근거로 경영진 리포트 문장 작성 (숫자는 못 지어내게 막음)

3. 저장 (**.md**) — `reports/monthly_sales_2026-05.md` (## 총평 / ## 카테고리 분석 / ## 다음 달 제언)

"숫자는 있는데 보고서가 없다"를 함수 한 번 호출로 해결합니다. 그리고 매월 1일 자동 실행되는 배치 잡으로 만들 수 있죠.

5. 백엔드 관점 — 배치/엔드포인트

이 에이전트는 두 가지 형태로 운영합니다.

[정기 배치] 매월 1일 새벽 → 리포트 자동 생성 → 슬랙/메일 발송

[온디맨드 API] GET /reports/monthly?month=2026-05 → 즉시 생성·반환

핵심은 **집계 함수와 서술 함수를 분리**해 두면, 배치 잡과 API 가 **같은 로직을 공유**한다는 점입니다. 입력(어느 달)과 출력(.md/JSON)만 정하면 cron 이든 웹 핸들러든 어디에나 꽃을 수 있죠. 이번 강에서 이 구조를 만듭니다.

1. "계산은 코드, 서술은 LLM" 원칙 (02번)
2. pandas로 핵심 수치 집계 (03번)
3. LLM에게 서술만 맡기기 — 환각 방지 (04번)
4. 리포트 저장 · 단일 진입점 (05번)
5. [신규] 예외 처리 — 데이터 누락 · 컬럼 변경 대응 (06번)
6. [신규] 차트로 시각화 더하기 (07번)

특히 신규 항목인 **예외 처리**(배치가 죽지 않게)와 **차트 시각화**(글+그림 한 세트)가 리포트를 실무 수준으로 끌어올립니다.

핵심 정리

- 경영진이 원하는 건 **표가 아니라 "읽을 수 있는 보고서"**.
- 매달 반복되는 **집계 → 해석 → 서술 → 저장**을 에이전트로 자동화.
- 핵심: **LLM은 계산기가 아니다** — 숫자는 코드, 서술만 LLM(산술 환각 방지).
- 목표물: CSV → 집계(pandas) → 서술(LLM) → **마크다운 리포트**.
- 백엔드로는 집계·서술 분리 → **배치·API가 같은 로직 공유**.

계산은 코드, 서술은 LLM

비즈니스 에이전트의 핵심 원칙입니다. **정확해야 하는 숫자는 코드(pandas)로 계산**해 사실(facts)로 확정하고, **LLM에는 그 사실을 근거로 글만 쓰게** 합니다. 이렇게 하면 보고서의 수치가 항상 정확하면서도 읽기 좋습니다.

1. 분업의 이유 — 각자 잘하는 일

LLM과 코드는 잘하는 게 다릅니다.

| | | | | |
|------------|---|---------------------|--|-------------------|
| 코드(pandas) | → | 계산: 정확 · 재현 가능 · 빠름 | | 서술: 못 함(딱딱한 숫자만) |
| LLM | → | 계산: 틀릴 수 있음(환각) | | 서술: 자연스러운 요약 · 해석 |

| 단계 | 담당 | 이유 |
|----------|------------|------------------------|
| 수치 집계 | pandas(코드) | 정확성·재현성 (LLM 산술 환각 방지) |
| 추세 해석·서술 | LLM | 자연어 요약·문장화가 강점 |
| 문서화 | 코드 | 저장·배포 자동화 |

각자 잘하는 일만 시킨다 — 이게 분업의 핵심입니다. 계산은 코드에게, 글쓰기는 LLM에게 맡기죠.

2. LLM에게 계산을 맡기면 생기는 일

"그냥 LLM에게 CSV 주고 다 시키면 안 되나?" 직접 해 보면 위험이 보입니다.

```
# 나쁜 예: LLM에게 계산까지 통째로 맡기기
from common import get_chat
import pandas as pd

csv_text = pd.read_csv("data/monthly_sales.csv").to_string()
llm = get_chat()
answer = llm.invoke(f"이 데이터로 총매출과 전월 대비 증감률을 계산해 보고서 써줘:\n{csv_text}").content
print(answer)
```

무엇이 위험한가

- "총매출 12,404,300원" → LLM이 "12,420,300원"으로 잘못 읽음 (환각)
- "전월 대비 27.1%" → 증감률 계산을 틀림 (큰 숫자 나눗셈)
- 그럴듯하지만 검증 안 된 숫자가 경영진 보고서에...

경영 보고서의 숫자가 틀리면 치명적입니다. 잘못된 매출 수치로 의사결정을 하면 실제 손실로 이어지죠. LLM 은 큰 숫자 계산을 머릿속으로 하다 틀리고, 더 나쁜 건 틀린 줄도 모르고 자신만만하게 답한다는 점입니다(환각).

3. 올바른 방법 — facts로 가두기

해결책은 숫자를 코드로 먼저 확정하고, LLM에게는 그 확정된 숫자(facts)만 주는 것입니다.

```
# 좋은 예: 숫자는 코드로 확정, LLM은 그 숫자로 글만
import pandas as pd

# 1) 계산은 코드 — 정확한 facts 생성
df = pd.read_csv("data/monthly_sales.csv")
facts = {
    "total": int(df.iloc[-1]["total"]),          # 정확한 총매출
    "growth_pct": round((df.iloc[-1]["total"] - df.iloc[-2]["total"])
                        / df.iloc[-2]["total"] * 100, 1), # 정확한 증감률
}

# 2) 서술은 LLM — facts만 주고 "새 숫자 금지"
prompt = (
    "아래 확정 수치만 근거로 보고서를 써라. 숫자를 새로 지어내지 마라.\n"
    f"총매출: {facts['total']:,}원, 전월 대비 {facts['growth_pct']}%"
)
```

핵심 두 가지:

1. 숫자는 코드(pandas)가 계산 → facts dict로 확정
2. 프롬프트에 facts를 박아 넣고 "새 숫자 지어내지 마라" 명시
→ LLM은 주어진 숫자로 '문장만' 만든다

이러면 보고서의 모든 숫자가 코드가 계산한 값과 정확히 일치합니다. LLM은 "12,404,300원이 전월 대비 늘었으니 회복세"라고 해석·서술만 하죠.

4. 프롬프트로 환각 차단하기

`write_report`의 프롬프트가 환각을 막는 장치입니다.

```
prompt = (
    "너는 승승장구몰의 경영기획 담당자다. 아래 '확정 수치'만 근거로 "
    "경영진 보고용 한국어 월간 매출 요약 리포트를 마크다운으로 작성하라. "
    "숫자를 새로 지어내지 말고 주어진 수치만 사용하라. " # ← 환각 차단 핵심
    "섹션: '## 총평 / ## 카테고리 분석 / ## 다음 달 제언'.\n\n"
    f"[확정 수치]\n"
    f"- 총매출: {facts['total']:,}원 (전월 대비 {facts['growth_pct']}%)\n"
    "# ... 나머지 facts ..."
)
```

세 가지 방어:

1. "확정 수치만 근거로" → 외부 지식 · 추측 금지
2. "숫자를 새로 지어내지 말고" → 산술 환각 차단
3. [확정 수치] 블록 → 쓸 수 있는 숫자를 명시적으로 제공

LLM에게 "재료(숫자)는 내가 줄 테니 너는 요리(글쓰기)만 해" 라고 역할을 한정하는 거죠.

5. 이 원칙이 적용되는 다른 곳

"계산은 코드, 서술은 LLM"은 비즈니스 리포트에만 쓰는 게 아닙니다. **정확한 사실 + 자연스러운 서술**이 모두 필요한 모든 곳에 적용됩니다.

- 데이터 분석 리포트: 통계는 코드, 인사이트 서술은 LLM (11강)
- 재무 요약: 합계 · 비율은 코드, 코멘트는 LLM
- A/B 테스트 결과: 유의성 검정은 코드, 결론 문장은 LLM
- 대시보드 자동 코멘트: 지표는 코드, "왜 이렇게 됐나"는 LLM

공통 패턴: **정확성이 중요한 부분은 코드, 표현력이 중요한 부분은 LLM**. 이 경계를 잘 긋는 게 신뢰할 수 있는 AI 시스템의 핵심입니다.

핵심 정리

- 원칙: **계산은 코드(pandas), 서술은 LLM** — 각자 잘하는 일만.
- LLM에게 계산을 맡기면 **산술 환각**으로 숫자가 틀릴 수 있음(경영 보고서엔 치명적).
- 해결: 숫자를 코드로 **facts**에 **확정** → 프롬프트에 박아 넣고 **"새 숫자 금지"**.
- 프롬프트 3종 방어: "확정 수치만"·"지어내지 마라"·"[확정 수치] 블록".
- 이 원칙은 **정확성+표현력**이 필요한 모든 리포트에 적용.

따라하기 — 핵심 수치 집계

목표

`monthly_sales.csv`에서 최신 달의 핵심 수치를 **pandas**로 정확히 계산해 facts(사실) 딕셔너리로 만듭니다. 이 facts가 04번에서 LLM 서술의 근거가 됩니다.

0. 실습 준비

```
conda activate agentic
pip install -U pandas          # 데이터 집계용 (11강에서 설치했다면 생략)
python code/common.py          # 환경/키 점검
```

- **pandas**: CSV 읽기·집계·정렬에 사용(11강에서 처음 등장).
- 데이터: `data/monthly_sales.csv` (month, 카테고리별 매출 컬럼들, total).

1. 임포트와 원칙 명시

`code/ch24_business.py`를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch24_business.py
import sys, pathlib, datetime
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, DATA

import pandas as pd

# [핵심 원칙] 계산은 코드(pandas), 서술은 LLM.
# [왜 분리] 숫자 계산을 LLM에게 맡기면 틀리거나 지어낼 수 있다(환각).
# 정확해야 하는 집계는 코드로 확정하고, LLM에는 '확정된 수치의 서술'만 맡긴다.
```

주석으로 원칙을 명시했습니다. 이 파일을 나중에 보는 사람이 "왜 계산을 코드로 하는지" 바로 알 수 있게요.

2. facts 집계 함수

핵심입니다. 최신 달을 골라 총매출·증감률·카테고리 1·2위를 코드로 정확히 계산합니다.

```
def load_facts() -> dict:
    """monthly_sales.csv에서 최신 달의 핵심 수치를 코드로 정확히 계산해 dict로 반환한다."""
    df = pd.read_csv(DATA / "monthly_sales.csv")
    df = df.sort_values("month").reset_index(drop=True) # 월 오름차순 정렬
    cur, prev = df.iloc[-1], df.iloc[-2] # 최신 달, 전월

    cat_cols = [c for c in df.columns if c not in ("month", "total")] # 카테고리 컬럼만
    cur_cats = cur[cat_cols].sort_values(ascending=False) # 카테고리별 매출 내림차순

    growth = (cur["total"] - prev["total"]) / prev["total"] * 100 # 전월 대비 증감률(%)
    return {
        "month": cur["month"],
        "prev_month": prev["month"],
        "total": int(cur["total"]),
        "prev_total": int(prev["total"]),
        "growth_pct": round(growth, 1),
        "top_category": cur_cats.index[0],
        "top_value": int(cur_cats.iloc[0]),
        "second_category": cur_cats.index[1],
        "second_value": int(cur_cats.iloc[1]),
        "by_category": {k: int(v) for k, v in cur_cats.items()},
    }
```

3. 한 줄씩 뜯어보기

3-1. 정렬해서 최신/전월 고르기

```
df = df.sort_values("month").reset_index(drop=True) # 월 오름차순 정렬
cur, prev = df.iloc[-1], df.iloc[-2] # 최신 달(-1), 전월(-2)
```

`sort_values("month")`로 월 순서를 보장한 뒤, `iloc[-1]` (맨 끝=최신), `iloc[-2]` (끝에서 둘째=전월)를 고릅니다. CSV 행 순서가 뒤섞여 있어도 정렬 덕에 항상 올바른 최신 달을 잡죠.

3-2. 카테고리 컬럼만 추리기

```
cat_cols = [c for c in df.columns if c not in ("month", "total")] # 카테고리만
cur_cats = cur[cat_cols].sort_values(ascending=False)           # 매출 내림차순
```

`month`와 `total`을 뺀 나머지가 카테고리 컬럼(전자기기·가전·식품...)입니다. 하드코딩하지 않고 동적으로 골라, 카테고리가 추가/변경돼도 코드를 안 고쳐도 됩니다. `sort_values(ascending=False)`로 매출 큰 순 정렬 → 1위·2위를 쉽게 뽑죠.

3-3. 증감률 계산

```
growth = (cur["total"] - prev["total"]) / prev["total"] * 100 # %
```

전월 대비 증감률 공식입니다. $(\text{이번달} - \text{전월}) / \text{전월} \times 100$. 이런 계산을 LLM에게 안 맡기고 코드로 하는 게 핵심이죠 — 정확하니까요.

3-4. int 변환과 by_category

```
"total": int(cur["total"]), # numpy 타입 → 파이썬 int (직렬화·포맷 안전)
"by_category": {k: int(v) for k, v in cur_cats.items()}, # 카테고리별 전체 매출
```

pandas는 `numpy.int64` 같은 타입을 쓰는데, `int()`로 파이썬 기본 `int`로 바꿉니다(JSON 직렬화·문자열 포맷 시 안전). `by_category`는 모든 카테고리의 매출을 dict로 담아, 04번 서술과 07번 차트에 서 재사용합니다.

4. 실행해 확인

```
if __name__ == "__main__":
    facts = load_facts()
    print("[집계된 핵심 수치]")
    print(f" 대상 월 : {facts['month']}")
    print(f" 총매출 : {facts['total']:},원 (전월 대비 {facts['growth_pct']}%)")
    print(f" 1위 : {facts['top_category']} {facts['top_value']:},원")
    print(f" 2위 : {facts['second_category']} {facts['second_value']:},원")
```

예상 출력

[집계된 핵심 수치]

대상 월 : 2026-05
총매출 : 12,404,300원 (전월 대비 27.1%)
1위 : 전자기기 4,449,850원
2위 : 가전 3,418,050원

{:,} 포맷으로 천단위 콤마를 찍어 읽기 좋게 했습니다. 이 숫자들이 모두 코드로 정확히 계산된 facts입니다. 04번에서 LLM이 이 숫자로 글을 쓰죠.

5. facts가 "단일 진실 공급원"

이 `load_facts()`가 만든 facts는 모든 곳에서 쓰는 단일 진실(single source of truth)입니다.

하나의 `facts`를 여러 곳에서 그대로 씁니다.

- `write_report()` — (04번) LLM 서술의 근거
- `save_report()` — (05번) 파일명·헤더
- `make_category_chart()` — (07번) 차트 데이터
- (API 응답) — JSON으로 그대로 반환 가능

숫자를 한 곳에서만 계산하니, 리포트·차트·API의 숫자가 항상 일치합니다. "계산은 한 번, 사용은 여러 곳"이 깔끔한 설계죠.

핵심 정리

- `load_facts()`: pandas 로 최신 달의 핵심 수치를 정확히 집계 → facts dict.
- `sort_values("month") + iloc[-1]/[-2]`로 최신/전월 안전하게 선택.
- 카테고리 컬럼을 동적으로 골라(하드코딩 X), 내림차순 정렬로 1:2 위 추출.
- 증감률 같은 계산은 코드로(LLM 에 안 맡김), `int()`로 타입 정리.
- facts 는 단일 진실 공급원 — 서술·저장·차트·API 가 모두 공유.

따라하기 — LLM 에게 서술 맡기기

목표

03번에서 집계한 facts를 LLM에게 주고, 숫자는 못 지어내게 막으면서 경영진 보고용 마크다운 리포트를 작성하게 합니다.

1. 서술 함수

`code/ch24_business.py`에 `write_report`를 추가합니다.

```
def write_report(facts: dict) -> str:
    """집계된 수치(facts)를 근거로 LLM이 경영진 보고용 마크다운 리포트를 작성한다.

    [왜 facts만 사용] 프롬프트에 '확정 수치'를 박아 넣고 새 숫자 금지를 지시해,
    LLM이 통계를 지어내지 못하게(환각 방지) 막는다.
    """
    llm = get_chat(temperature=0.3) # 서술의 자연스러움 위해 약간 높임
    cat_lines = "\n".join(f"- {k}: {v:,.}원" for k, v in facts["by_category"].items())
    prompt = (
        "너는 승승장구몰의 경영기획 담당자다. 아래 '확정 수치'만 근거로 "
        "경영진 보고용 한국어 월간 매출 요약 리포트를 마크다운으로 작성하라. "
        "숫자를 새로 지어내지 말고 주어진 수치만 사용하라. "
        "섹션: '## 총평 / ## 카테고리 분석 / ## 다음 달 제언'.\n\n"
        f"[확정 수치]\n"
        f"- 대상 월: {facts['month']} (전월: {facts['prev_month']})\n"
        f"- 총매출: {facts['total']:,.}원 (전월 {facts['prev_total']:,.}원, "
        f"증감 {facts['growth_pct']}%)\n"
        f"- 최대 카테고리: {facts['top_category']} {facts['top_value']:,.}원\n"
        f"- 2위 카테고리: {facts['second_category']} {facts['second_value']:,.}원\n"
        f"[카테고리별 매출]\n{cat_lines}\n"
    )
    return llm.invoke(prompt).content
```

2. 설계 포인트 셋

2-1. temperature=0.3 — 서술은 약간의 자연스러움

```
llm = get_chat(temperature=0.3) # 서술의 자연스러움 위해 약간 높임
```

분류·계산(0.0)과 달리, 서술은 **0.3** 정도로 약간 높입니다. 너무 0이면 문장이 기계적이고, 너무 높으면 사실에서 벗어나죠. "정확한 숫자 + 자연스러운 문장"의 균형점이 0.3 근처입니다. 숫자는 어차피 **facts**로 고정돼 있으니, temperature를 올려도 숫자는 안 흔들립니다.

2-2. facts를 프롬프트에 박아 넣기

```
f"- 총매출: {facts['total']:,}원 (전월 {facts['prev_total']:,}원, 증감 {facts['growth_pct']}%)\\n"
```

LLM이 쓸 수 있는 모든 숫자를 프롬프트에 명시합니다. `{:,}`로 천단위 콤마까지 미리 포맷해서 주죠. LLM은 이 숫자를 그대로 가져다 문장에 넣기만 하면 됩니다 — 계산할 필요가 없어요.

2-3. 카테고리 목록 문자열화

```
cat_lines = "\\n".join(f"- {k}: {v:,}원" for k, v in facts["by_category"].items())
```

`by_category` dict를 `"- 전자기기: 4,449,850원\\n- 가전: 3,418,050원..."` 같은 마크다운 목록 문자열로 변환합니다. LLM이 카테고리 분석 섹션을 쓸 때 이 목록을 근거로 삼죠.

3. 실행해 보기

```
if __name__ == "__main__":
    facts = load_facts()
    print("[LLM 리포트 생성 중...]")
    body = write_report(facts)
    print("=" * 60)
    print(body)
    print("=" * 60)
```

예상 출력

[LLM 리포트 생성 중...]

총평

2026년 5월 총매출은 12,404,300원으로 전월(9,760,300원) 대비 27.1% 증가하며
뚜렷한 회복세를 보였습니다. 전 카테고리에 걸쳐 고른 성장이 나타났습니다.

카테고리 분석

- 전자기기가 4,449,850원으로 전체 매출을 견인했고, 가전(3,418,050원)이
그 뒤를 이었습니다. 두 카테고리가 전체의 절반 이상을 차지합니다.
- 식품·패션의류는 안정적 기여를 유지했습니다.

다음 달 제언

전자기기·가전의 호조를 이어가기 위해 해당 카테고리 프로모션을 집중하고,
상대적으로 비중이 낮은 카테고리는 ... 강화를 검토할 필요가 있습니다.

관찰 포인트: 보고서의 모든 숫자(**12,404,300원**, **27.1%**, **4,449,850원**)는 **facts**와 정확히 일치합니다. LLM은 그 숫자를 "회복세", "전체를 견인", "절반 이상" 같은 해석·서술로 감쌌을 뿐이죠. 이게 "계산은 코드, 서술은 LLM"의 결과물입니다.

4. 숫자가 안 틀리는지 검증해 보기

정말 LLM이 숫자를 안 지어내는지 확인하려면, 리포트의 숫자가 facts에 있는지 대조합니다.

```
body = write_report(facts)
# 리포트에 등장한 핵심 숫자가 facts와 일치하는지 간단 확인
assert f"{facts['total']:,}" in body, "총매출이 리포트에 정확히 안 나옴!"
print("✅ 총매출 수치 일치 확인:", f"{facts['total']:,}원")
```

예상 출력

✅ 총매출 수치 일치 확인: 12,404,300원

프롬프트로 "새 숫자 금지"를 명시했으니, 리포트의 총매출은 facts의 값과 정확히 같습니다. 이런 검증 단계를 배치에 넣으면, 혹시 LLM이 숫자를 틀렸을 때 잡아낼 수 있죠(16강 검증 정신).

5. 서술의 품질을 높이는 팁

리포트 품질을 더 높이려면 프롬프트에 맥락을 더 줄 수 있습니다.

```
# 예: 비즈니스 맥락을 추가하면 서술이 풍부해짐
prompt += (
    "\n[참고 맥락]\n"
    "- 승승장구몰은 종합 온라인 쇼핑몰이다.\n"
    "- 5월은 가정의 달 특수가 있는 시즌이다.\n"
)
```

이렇게 하면 LLM이 "가정의 달 특수로 전자기기·가전 선물 수요가 늘어..." 같은 맥락 있는 해석을 합니다. 단, 맥락은 사실이어야 하고, 숫자는 여전히 facts에서만 가져오게 해야 하죠. 서술의 풍부함과 정확성을 함께 잡는 균형입니다.

핵심 정리

- write_report(): facts 를 프롬프트에 박아 넣고 **LLM 이 서술만** 하게.
- temperature=0.3 — 서술의 자연스러움(숫자는 facts 로 고정돼 안 흔들림).
- 모든 숫자를 미리 포맷({:,.})해 제공 → LLM 은 가져다 쓰기만.
- "새 숫자 금지" 명시로 산술 환각 차단 → 리포트 숫자 = facts 숫자.
- assert 로 숫자 일치를 검증하면 배치 안정성 ↑.
- 맥락을 더하면 서술이 풍부해짐(단, 숫자는 facts 에서만).

따라하기 — 리포트 저장

목표

서술된 리포트를 월별 파일명으로 저장하고, 집계→서술→저장 전체를 단일 진입점 함수로 묶어 배치·API에서 한 번에 호출할 수 있게 만듭니다.

1. 저장 함수

code/ch24_business.py 에 save_report 를 추가합니다.

```
def save_report(facts: dict, body: str) -> pathlib.Path:
    """리포트를 reports/ 폴더에 월별 파일명으로 저장하고 경로를 반환한다."""
    out_dir = pathlib.Path(__file__).resolve().parent.parent / "reports"
    out_dir.mkdir(exist_ok=True)      # reports/ 폴더가 없으면 생성
    path = out_dir / f"monthly_sales_{facts['month']}.md"
    header = (f"# {facts['month']} 월간 매출 보고서\n\n"
              f"> 생성일: {datetime.date.today().isoformat()} (자동 생성)\n\n")
    path.write_text(header + body, encoding="utf-8")
    return path
```

짚어 둘 점

- `f"monthly_sales_{facts['month']}.md"` — 월별 파일명(`monthly_sales_2026-05.md`). 달마다 다른 파일이라 덮어쓰이지 않고 누적됩니다.
- `header` — 제목·생성일을 본문 위에 붙여 완성된 문서로.
- `encoding="utf-8"` — 한글 보존(22·23강과 동일 원칙).
- 22강 리서치 리포트와 같은 패턴이지만, 파일명이 월 기준이라 매월 리포트가 차곡차곡 쌓이죠.

2. 단일 진입점 — `generate_monthly_report`

집계→서술→저장을 한 함수로 묶습니다. 배치·API는 이 함수 하나만 부르면 됩니다.

```
def generate_monthly_report() -> pathlib.Path:
    """집계 → 서술 → 저장 전체 파이프라인. 배치/엔드포인트에서 이 함수 하나만 호출하면 된다.

    [왜 단일 진입점] 매월 1일 자동 실행 같은 배치는 이 함수 하나만 부르면 끝나도록
    파이프라인을 한 함수로 묶어 운영을 단순화한다.
    """
    facts = load_facts()          # 1) 코드로 정확히 집계
    body = write_report(facts)    # 2) LLM이 서술
    return save_report(facts, body) # 3) .md 저장
```

세 함수(`load_facts` / `write_report` / `save_report`)를 순서대로 호출할 뿐입니다. 하지만 이 한 함수로 묶는 것이 운영을 단순하게 만들죠.

```
[배치 스케줄러]  매월 1일 → generate_monthly_report() ← 이 한 줄이면 끝
[API 핸들러]     GET /reports/monthly → generate_monthly_report()
```


cron이든 웹 핸들러든 `generate_monthly_report()` 한 줄만 부르면 리포트가 생성됩니다. 내부 단계 (집계·서술·저장)는 신경 안 써도 되죠.

3. 전체 실행

```
if __name__ == "__main__":
    facts = load_facts()
    print("[집계된 핵심 수치]")
    print(f" 대상 월 : {facts['month']}")
    print(f" 총매출 : {facts['total']:,}원 (전월 대비 {facts['growth_pct']}%)")
    print(f" 1위 : {facts['top_category']} {facts['top_value']:,}원")

    print("\n[LLM 리포트 생성 중...]")
    body = write_report(facts)
    path = save_report(facts, body)

    print("=" * 60)
    print(body)
    print("=" * 60)
    print("[저장 완료]", path)
```

예상 출력

```
[집계된 핵심 수치]
대상 월 : 2026-05
총매출 : 12,404,300원 (전월 대비 27.1%)
1위 : 전자기기 4,449,850원

[LLM 리포트 생성 중...]

==
## 총평
2026년 5월 총매출은 12,404,300원으로 전월 대비 27.1% 증가하며 ...
## 카테고리 분석
- 전자기기가 4,449,850원으로 전체를 견인했고 ...
## 다음 달 제언
전자기기 · 가전의 호조를 ...

==
[저장 완료] /Users/.../reports/monthly_sales_2026-05.md
```

4. 저장된 파일 확인

```
cat reports/monthly_sales_2026-05.md
```

```
# 2026-05 월간 매출 보고서
```

```
> 생성일: 2026-06-07 (자동 생성)
```

```
## 총평
```

```
2026년 5월 총매출은 12,404,300원으로 ...
```

```
## 카테고리 분석
```

```
...
```

```
## 다음 달 제언
```

```
...
```

제목·생성일 헤더 + 본문이 합쳐진 완성된 경영 보고서입니다. 이대로 슬랙에 올리거나, HTML/PDF로 변환하거나, 메일로 보낼 수 있죠.

5. 운영 안정성 더하기 (예고)

이 기본 파이프라인을 실무에 올리려면 방어 로직이 더 필요합니다.

- 집계 단계 예외: 파일 없음 · 컬럼 변경 → 06번에서 다룸
- 생성 실패 시: 직전 달 리포트로 폴백하거나 알림 발송
- 같은 달 재요청: 캐싱으로 저장된 리포트 반환 (29강)

운영 팁: `generate_monthly_report()` 한 함수만 cron(매월 1일)이나 API 핸들러에 연결하면 됩니다. 집계 단계에서 예외(파일 없음·컬럼 변경)를 잡아 로그를 남기고, 생성 실패 시 직전 달 리포트로 폴백하거나 알림을 보내는 방어 로직을 더하면 운영 안정성이 올라갑니다.

예외 처리 — 데이터 누락·컬럼 변경 대응

load_facts 는 "완벽한 CSV"를 전제로 합니다.

파일이 없거나(FileNotFoundError), 컬럼이 빠지거나(KeyError), 데이터가 비면 그대로 터집니다. 매월 자동 실행되는 배치는 멈추면 안 되므로, 안전하게 감싼 load_facts_safe 로 죽지 않고 친절히 대응하게 만듭니다.

1. 완벽한 데이터는 없다

03번 load_facts 는 데이터가 항상 완벽하다고 가정합니다. 하지만 현실은:

- 데이터팀이 CSV를 안 올림 → FileNotFoundError
- 컬럼명을 'total' → 'sum'으로 바꿈 → KeyError
- 이번 달 데이터가 아직 안 들어옴 → 빈 데이터프레임
- CSV가 깨짐(인코딩 · 포맷) → ParserError

이런 일이 생기면 load_facts 는 예외를 던지고 죽습니다. 매월 1일 새벽 배치가 돌다가 데이터 한 번 잘못됐다고 멈추면? 담당자가 새벽에 호출당하죠. 배치는 어지간한 문제에는 죽지 않고 넘어가야 합니다.

2. 안전판의 철학 — 절대 예외를 던지지 않는다

load_facts_safe 의 핵심 약속: 어떤 상황에서도 dict를 반환한다(절대 예외를 던지지 않는다).

성공: load_facts 결과 + {"ok": True}
실패: 기본값 + {"ok": False, "error": "사람이 읽을 안내 메시지"}

호출하는 쪽은 facts["ok"] 하나만 보면 성공/실패를 분기할 수 있죠. 예외를 try/except로 잡는 대신, "ok 플래그"로 결과를 받는 패턴입니다.

```
# 실행: python "sections/24_business_agent/06_예외처리-데이터-누락-대응.py"
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat, DATA
import pandas as pd
from ch24_business import load_facts # 원본 집계 로직 재사용
```

```
# [핵심] '계산은 코드'라는 원칙은 그대로 두되, 코드가 죽지 않게 방어막을 씌운다.  
REQUIRED_COLS = ["month", "total"] # 집계에 반드시 있어야 하는 컬럼
```

추가 설치 라이브러리는 **없습니다**(pandas는 03번에서 설치).

3. 실패 시 공통 반환 — fail()

먼저 실패할 때 돌려줄 기본값 **dict**를 만드는 헬퍼입니다.

```
def load_facts_safe(csv_path: pathlib.Path = None) -> dict:  
    """load_facts 를 try/except 로 감싼 안전판. 어떤 상황이든 dict 를 반환한다."""  
    path = csv_path or (DATA / "monthly_sales.csv")  
  
    def fail(msg: str) -> dict:  
        """실패 시 공통 반환 — 호출부가 facts['ok'] 만 보면 분기할 수 있게 한다."""  
        return {  
            "ok": False,  
            "error": msg,  
            "month": "N/A", "prev_month": "N/A",  
            "total": 0, "prev_total": 0, "growth_pct": 0.0,  
            "top_category": "없음", "top_value": 0,  
            "second_category": "없음", "second_value": 0,  
            "by_category": {},  
        }  
    
```

`fail()` 은 `load_facts`와 똑같은 키 구조에 기본값(0, "N/A", "없음")을 채워 돌려줍니다. 이러면 실패해도 호출부가 `facts["total"]` 같은 키를 안전하게 읽을 수 있죠(KeyError 안 남). `error`에는 사람이 읽을 안내 메시지를 담습니다.

4. 단계별 검증 — 터지기 전에 친절히 막기

이제 예외가 나기 전에 미리 확인해, 친절한 메시지로 막습니다.

```
# 1) 파일 존재 확인 — pandas 가 던지기 전에 먼저 친절히 안내  
if not path.exists():  
    return fail(f"매출 파일을 찾지 못했습니다: {path} (경로/파일명을 확인하세요)")
```

```

# 2) 읽기 시도 (깨진 CSV · 인코딩 오류 대비)
try:
    df = pd.read_csv(path, encoding="utf-8-sig") # BOM 제거 위해 utf-8-sig
except Exception as e:
    return fail(f"CSV 를 읽을 수 없습니다: {type(e).__name__} - {e}")

# 3) 빈 데이터 확인
if df.empty:
    return fail("매출 데이터가 비어 있습니다(행 0개).")

# 4) 필수 컬럼 확인
missing = [c for c in REQUIRED_COLS if c not in df.columns]
if missing:
    return fail(f"필수 컬럼이 누락되었습니다: {missing} / 현재 컬럼: {list(df.columns)}")

# 5) 행 수 확인 (전월 대비는 최소 2개월 필요)
if len(df) < 2:
    return fail(f"전월 대비 계산에는 최소 2개월이 필요한데 {len(df)}개월뿐입니다.")

```

각 단계가 하나의 실패 원인을 미리 차단합니다.

1. 파일 없음 → "파일을 찾지 못했습니다" (경로 확인 안내)
2. 읽기 실패 → "CSV를 읽을 수 없습니다" (오류 종류 포함)
3. 빈 데이터 → "데이터가 비어 있습니다"
4. 컬럼 누락 → "필수 컬럼이 누락" + 현재 컬럼 목록(디버깅 도움)
5. 행 부족 → "최소 2개월 필요" (전월 대비 계산 조건)

특히 "현재 컬럼: [...]" 처럼 실패 메시지에 진단 정보를 넣는 게 실무 팁입니다. 담당자가 로그만 보고 "아, total을 sum으로 바꿨구나" 바로 알 수 있죠.

5. 검증 통과 후 실제 집계

모든 검증을 통과하면, 실제 집계는 검증된 원본 로직에 위임합니다.

```

# 6) 실제 집계 (마지막 그물망 try/except)
try:
    facts = load_facts() # 검증된 원본 로직 재사용
except Exception as e:
    return fail(f"집계 중 알 수 없는 오류: {type(e).__name__} - {e}")

```

```
facts["ok"] = True          # 성공 표시
return facts
```

검증을 다 했어도 예상 못한 오류가 있을 수 있으니, 마지막 try/except로 그물망을 한 번 더 칩니다. "절대 예외를 던지지 않는다"는 약속을 지키는 거죠.

6. 4가지 상황 시연

일부러 문제 상황을 만들어 죽지 않고 넘어가는지 확인합니다.

```
def print_facts(label: str, facts: dict):
    if facts["ok"]:
        print(f" 상태      : 정상 / 대상 월 {facts['month']} / 총매출 {facts['total']:,.}원")
    else:
        print(f" 상태      : 실패(기본값 반환) / 안내: {facts['error']}")

if __name__ == "__main__":
    print("[1] 정상 파일")
    print_facts("정상", load_facts_safe())

    print("[2] 없는 경로")
    print_facts("없는파일", load_facts_safe(DATA / "없는파일.csv"))

    # [3] total 컬럼 누락한 임시 CSV
    bad = DATA / "_tmp_나쁜컬럼.csv"
    pd.read_csv(DATA / "monthly_sales.csv", encoding="utf-8-sig").drop(
        columns=["total"]).to_csv(bad, index=False, encoding="utf-8-sig")
    print("[3] total 컬럼 누락")
    print_facts("컬럼누락", load_facts_safe(bad))
    bad.unlink(missing_ok=True)  # 임시 파일 정리

    # [4] 빈 데이터(헤더만)
    empty = DATA / "_tmp_빈데이터.csv"
    pd.DataFrame(columns=["month", "total"]).to_csv(empty, index=False, encoding="utf-8-sig")
    print("[4] 빈 데이터")
    print_facts("빈데이터", load_facts_safe(empty))
    empty.unlink(missing_ok=True)
```

예상 출력

```
[1] 정상 파일
    상태   : 정상 / 대상 월 2026-05 / 총매출 12,404,300원
[2] 없는 경로
    상태   : 실패(기본값 반환) / 안내: 매출 파일을 찾지 못했습니다: .../없는파일.csv ...
[3] total 컬럼 누락
    상태   : 실패(기본값 반환) / 안내: 필수 컬럼이 누락되었습니다: ['total'] / 현재 컬럼: [...]
[4] 빈 데이터
    상태   : 실패(기본값 반환) / 안내: 매출 데이터가 비어 있습니다(행 0개).
```

4가지 문제 상황 모두 예외로 멈추지 않고 dict를 돌려줬습니다. 배치는 끝까지 완주하고, 실패한 건은 `error` 메시지로 로그에 남죠. 임시 파일은 `unlink(missing_ok=True)`로 정리합니다.

7. 호출부에서 활용

`load_facts_safe`를 쓰면 배치 코드가 깔끔해집니다.

```
facts = load_facts_safe()
if not facts["ok"]:
    # 실패: 로그 남기고, 직전 달 리포트로 폴백하거나 알림 발송
    print(f"[배치 경고] 리포트 생성 실패: {facts['error']}")
    notify_admin(facts["error"])    # 담당자에게 슬랙 알림
else:
    # 성공: 정상 리포트 생성
    body = write_report(facts)
    save_report(facts, body)
```

`facts["ok"]` 하나로 분기합니다. 실패하면 알림만 보내고 배치는 계속 돌죠. 새벽에 배치 전체가 죽어 호출당하는 일이 없어집니다.

핵심 정리

- `load_facts`는 완벽한 CSV 전제 → 파일 없음·컬럼 변경·빈 데이터에 그대로 터짐.
- `load_facts_safe`: 절대 예외를 안 던지고 항상 dict 반환(ok 플래그).
- 실패 시 `fail()`로 같은 키 구조 + 기본값 + 안내 메시지 반환.
- 단계별 검증(파일·읽기·빈데이터·컬럼·행수) + 마지막 그물망 `try/except`.
- 실패 메시지에 진단 정보(현재 컬럼 목록 등)를 넣어 디버깅 도움.
- 배치는 `facts["ok"]`로 분기 → 실패해도 알림만 보내고 계속(견고성).

차트로 시각화 더하기

월간 리포트가 글(마크다운)뿐이면 한눈에 안 들어옵니다. 집계된 facts 로 카테고리별 매출 막대그래프를 png 로 그려 저장하고, 리포트 본문 맨 위에 이미지 링크를 끼워 넣어 "글 + 그림 한 세트"로 만듭니다. 경영진이 한눈에 보죠.

1. 글만으로는 부족하다

04번 리포트는 텍스트뿐입니다.

```
## 카테고리 분석
- 전자기기가 4,449,850원으로 전체를 견인했고, 가전(3,418,050원)이 ...
```

들린 건 아니지만, 숫자 나열은 한눈에 안 들어옵니다. 경영진은 "어느 카테고리가 큰지"를 그림 한 장으로 보고 싶어 하죠. 막대그래프 하나면 카테고리 간 크기 비교가 즉각 보입니다. 그래서 차트를 더합니다.

2. matplotlib 준비

```
pip install -U matplotlib koreanize-matplotlib # 차트 + 한글 폰트(OS 무관)
```

```
# 실행: python "sections/24_business_agent/07_차트로-시각화-더하기.py"
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat, DATA

import matplotlib
matplotlib.use("Agg") # [왜] 화면 없는 서버/배치에서도 png 저장이 되도록 비대화형 백엔드
import matplotlib.pyplot as plt
import koreanize_matplotlib # 한글 폰트 설정(OS 무관) — import 한 줄로 끝

from ch24_business import load_facts, write_report, save_report # 집계 · 서술 · 저장 재사용

REPORTS_DIR = pathlib.Path(__file__).resolve().parents[2] / "reports"
```


중요한 두 가지 설정

- `matplotlib.use("Agg")` — 화면 없는 서버/배치에서도 png가 저장되게 하는 **비대화형 백엔드**. `import matplotlib.pyplot` 전에 호출해야 합니다. 이게 없으면 화면 없는 환경에서 에러가 나죠.
- `import koreanize_matplotlib` — 한글 폰트를 OS 무관하게 한 줄로 설정. 안 하면 한글이 □□□로 깨집니다. 번들된 나눔고딕을 적용하고 음수 부호 깨짐까지 자동으로 잡아 주죠(맥·윈도우·리눅스·코랩 동일). `matplotlib.pyplot`을 import 한 다음 줄에 둡니다.

3. 차트 그리기 함수

facts의 `by_category`로 막대그래프를 그립니다. 숫자는 이미 확정된 **facts**를 그대로 쓰니, 차트도 "계산은 코드" 원칙을 따르죠(환각 없음).

```
def make_category_chart(facts: dict) -> pathlib.Path:
    """facts['by_category']로 카테고리별 매출 막대그래프를 그려 png로 저장한다."""
    REPORTS_DIR.mkdir(exist_ok=True)
    cats = list(facts["by_category"].keys())           # 카테고리명
    vals = [v / 10000 for v in facts["by_category"].values()] # 만원 단위로 환산(가독성)

    fig, ax = plt.subplots(figsize=(9, 5))
    bars = ax.bar(cats, vals, color="#4C72B0")
    ax.bar(cats[0], vals[0], color="#DD8452")         # 1위 카테고리만 강조 색(by_category는 내림차순)
    ax.set_title(f"승승장구몰 {facts['month']} 카테고리별 매출")
    ax.set_ylabel("매출(만원)")
    ax.set_xlabel("카테고리")
    plt.xticks(rotation=30, ha="right")               # 카테고리명이 길어 비스듬히
    for b, v in zip(bars, vals):                       # 막대 위에 값 라벨
        ax.text(b.get_x() + b.get_width() / 2, v, f"{v:,.0f}",
                ha="center", va="bottom", fontsize=9)
    plt.tight_layout()

    path = REPORTS_DIR / f"chart_{facts['month']}.png"
    plt.savefig(path, dpi=120) # [경고] plt.show() 금지 — 배치에서는 저장만 한다
    plt.close(fig)           # 메모리 누수 방지(반복 생성 대비)
    return path
```

짚어 둘 점

- `v / 10000` — 원 단위는 숫자가 너무 커서 **만원** 단위로 환산(축 가독성).
- `ax.bar(cats[0], ...)` 강조 색 — `by_category`가 내림차순이라 `cats[0]`이 1위. 다른 색으로 칠해 1위를 강조.

- `ax.text(...)` — 막대 위에 값 라벨을 찍어 정확한 수치도 보이게.
- `plt.savefig(...)` + `plt.close(fig)` — 저장만 하고 닫습니다. `plt.show()` 는 배치에서 금물(화면이 없으니 멈춤). `close` 로 메모리 누수도 방지.

4. 리포트에 이미지 링크 삽입

차트 png를 리포트 본문 맨 위에 마크다운 이미지 링크로 끼워 넣습니다.

```
def write_report_with_chart(facts: dict, body: str, chart_path: pathlib.Path) -> str:
    """리포트 본문 맨 위에 차트 이미지 링크를 삽입한다.

    [왜 상대경로] .md 와 png 가 같은 reports/ 폴더에 있으므로 파일명만 적으면
    마크다운 뷰어가 바로 이미지를 띄운다. (절대경로는 다른 PC에서 깨진다)
    """

    img_md = f"![{facts['month']} 매출차트]({chart_path.name})\n\n"
    return img_md + body
```

! [설명](파일명.png) 이 마크다운 이미지 문법입니다. 핵심은 `chart_path.name` (파일명만) — 절대경로가 아니라 상대경로를 씁니다. `.md` 와 `.png` 가 같은 `reports/` 폴더에 있으니, 파일명만 적으면 어느 PC에서 열어도 이미지가 뜨죠. 절대경로(`/Users/macro/...`)를 쓰면 다른 PC에서 깨집니다.

5. 전체 실행

```
if __name__ == "__main__":
    print("[1] 핵심 수치 집계 (코드)")
    facts = load_facts()
    print(f" 1위 : {facts['top_category']} {facts['top_value']:,}원")

    print("[2] 카테고리별 매출 차트 생성 (png 저장)")
    chart_path = make_category_chart(facts)
    print(f" 저장됨 : {chart_path}")

    print("[3] LLM 서술 + 차트 링크 삽입 → .md 저장")
    body = write_report(facts) # LLM 서술 재사용
    body_with_img = write_report_with_chart(facts, body, chart_path)
    md_path = save_report(facts, body_with_img) # 저장 로직 재사용
    print(f" 저장됨 : {md_path}")
```

예상 출력

```
[1] 핵심 수치 집계 (코드)
    1위 : 전자기기 4,449,850원
[2] 카테고리별 매출 차트 생성 (png 저장)
    저장됨 : /Users/.../reports/chart_2026-05.png
[3] LLM 서술 + 차트 링크 삽입 → .md 저장
    저장됨 : /Users/.../reports/monthly_sales_2026-05.md
```

생성된 `.md`를 마크다운 뷰어(VS Code·GitHub·노션)로 열면, 맨 위에 막대그래프가 뜨고 그 아래 서술이 이어집니다.

```
# 2026-05 월간 매출 보고서

![2026-05 매출차트](chart_2026-05.png)    ← 그림이 여기 렌더링됨

## 총평
2026년 5월 총매출은 ...
```

글 + 그림 한 세트가 완성됐습니다. 경영진은 그림으로 큰 흐름을, 글로 해석을 한눈에 보죠.

6. 차트도 "계산은 코드" 원칙

여기서 다시 강조할 점 — 차트의 숫자도 **facts**에서 가져옵니다.

```
facts['by_category'] → 막대 높이 (코드가 계산한 정확한 값)
                     → 글(LLM 서술)과 그림(차트)이 같은 숫자를 공유
```

리포트의 글과 차트가 같은 **facts**를 쓰니, "글에선 전자기기 1위인데 그림에선 가전이 높네?" 같은 불일치가 절대 안 생깁니다. 03번에서 강조한 "facts는 단일 진실 공급원"의 가치가 여기서도 빛나죠. 차트도 LLM이 아니라 코드가 그리니 환각이 없습니다.

실습문제

문제 1 — 카테고리별 성장률 분석 + 차트

`code/ch24_business.py`를 복사해 `code/ch24_ex_growth.py`를 만드세요. `load_facts`를 확장해 각 카테고리의 첫 달 대비 마지막 달 성장률(%)을 계산하고, matplotlib로 카테고리별 최신 달 매출 막대그래프를 `reports/category_sales.png`로 저장하세요.

- 사용 파일: `data/monthly_sales.csv`
- 기대 결과: 콘솔에 성장률 상위/하위 카테고리명이 출력되고 `reports/category_sales.png`가 생성된다.

문제 2 — 특정 달 지정 리포트

`generate_monthly_report(month="2026-03")`처럼 특정 달을 지정해 리포트를 생성하도록 인자를 추가하고, 존재하지 않는 달이면 친절한 예외 메시지를 출력하게 하세요.

- 검증: 존재하는 달과 없는 달(예: "1999-01")을 각각 호출해, 정상 리포트와 예외 메시지가 구분되어 출력되는지 확인.

3. deployment

내 PC에선 됐는데요

지금까지의 실습 코드는 **데모**입니다.

키가 코드에 박혀 있고, 실패하면 그냥 죽고, 무슨 일이 있었는지 로그도 없죠.

이대로 서버에 올리면 운영에서 바로 사고가 납니다.

백엔드 개발자라면 익숙한 그 격언 — "**내 PC에선 됐는데요**" — 를 막는 게 이번 강입니다.

1. 데모 코드 vs 운영 코드

지금까지 우리가 짠 코드를 떠올려 봅시다.

```
# 지금까지의 데모 코드 (문제투성이)
client = genai.Client(api_key="AIzaSy...")
resp = llm.invoke("무료배송 기준?")
print(resp.content)
```

① 키가 코드에 박힘
② 실패하면? 그냥 죽음
③ 무슨 일이 있었는지 기록 없음

이건 "**내 PC에서 한 번 돌려보는**" 데는 충분합니다. 하지만 실제 사용자에게 서비스하려면 치명적입니다.

- ① 키가 코드에 박힘 → Git에 올리면 키 유출! 환경 바뀌면 코드 수정해야
- ② 실패하면 죽음 → LLM이 잠깐 느리거나 rate limit 걸리면 서비스 중단
- ③ 로그 없음 → "왜 안 됐어?"에 답할 수 없음 (운영의 눈이 없음)

2. "내 PC에선 됐는데요"의 정체

개발자라면 한 번쯤 들어본 말입니다. 내 컴퓨터에선 분명 잘 돌아갔는데, 서버에 올리니 안 되는 상황.

내 PC: 키가 내 환경변수에 있음, 네트워크 좋음, 나 혼자 씀
서버: 키 설정 안 됨, 네트워크 불안정, 수십 명이 동시에 씀
→ 같은 코드인데 여기서 터짐

원인은 대부분 환경 차이와 예외 미처리입니다.

내 PC라는 특수 환경에 코드가 의존하고 있었던 거죠.

운영은 어디서 돌려도 똑같이, 문제가 생겨도 안 죽게 만들어야 합니다.

3. 운영 가능한 에이전트의 4대 요소

데모를 운영 코드로 바꾸려면 네 가지를 갖춰야 합니다.

- ① 설정 분리 비밀(키) · 설정(모델 · 온도)을 코드 밖으로 → 어디서나 동일 동작
- ② 로깅 print 대신 구조적 로그 → 무슨 일이 있었는지 추적 가능
- ③ 예외/재시도 일시 오류엔 재시도, 끝내 실패해도 안내 메시지 → 안 죽음
- ④ 비용 관리 모델 선택 · 캐싱 → 무료티어 한도 · 속도 관리

모델만 좋다고 서비스가 되는 게 아닙니다. 이 4가지가 운영 품질을 만듭니다. 같은 LLM을 쓰더라도, 이 4가지를 갖춘 시스템은 안정적이고 추적 가능하고 저렴합니다.

4. 이번 강의 흐름

29강은 이 4대 요소를 차례로 구현합니다.

- 02번: 설정 분리 .env(비밀) vs config.yaml(설정)
- 03번: 구조적 로깅 logging 으로 콘솔+파일 기록
- 04번: 예외/재시도 지수 백오프 재시도 + 폴백
- 05번: 비용 · 지연 모델 선택 · 프롬프트 · 캐싱 · top_k
- 06번: 따라하기 위 셋을 묶은 '견고한 운영 래퍼'
- 07번: 따라하기 FastAPI 엔드포인트로 감싸기
- 08번: 신규 응답 캐싱으로 비용 줄이기

이론(02~05)을 먼저 보고, 06번에서 견고한 래퍼 코드로 묶습니다. 그다음 07번에서 HTTP API로 노출하고, 08번에서 캐싱으로 비용을 줄이죠.

5. 핵심 철학 — 비즈니스 로직과 운영을 분리

이번 강을 관통하는 생각입니다.

```
[에이전트 본체]  "무엇을 답하나" (비즈니스 로직)
+
[운영 계층]      설정 · 로깅 · 재시도 · HTTP (어떻게 안정적으로 서비스하나)
```

핵심 정리

- 지금까지의 코드는 **데모** — 키 박제·실패 시 죽음·로그 없음.
- "내 PC에선 됐는데요" = 환경 차이 + 예외 미처리. 운영은 어디서나 동일·안 죽게.
- 운영 4대 요소: ① **설정 분리** ② **로깅** ③ **예외/재시도** ④ **비용 관리**.
- 모델이 좋아도 이 4가지가 없으면 서비스가 안 됨 — 이게 **운영 품질**.
- 핵심 철학: **비즈니스 로직(본체)**과 **운영 계층**을 분리한다.

설정 분리 — .env vs config.yaml

운영의 첫걸음은 **하드코딩 제거**입니다. 코드에 박힌 키·모델명·온도를 밖으로 빼되, 두 곳으로 나눕니다: 비밀(API 키)은 **.env**, 행동을 바꾸는 설정(모델명·온도·재시도 횟수)은 **config.yaml**. 관리 주체가 다르기 때문입니다.

1. 하드코딩이 왜 문제인가

값을 코드에 직접 박는 걸 **하드코딩**이라 합니다.

```
# 하드코딩 (나쁨)
client = genai.Client(api_key="AIzaSy...") # 키가 코드에
llm = get_chat(temperature=0.2)          # 온도가 코드에
```

문제점:

- 키 유출: Git에 올리면 누구나 키를 봄 → 도용 · 과금 폭탄
- 환경 전환 불가: dev는 0.2, prod는 0.0으로? → 코드를 고쳐야 함
- 협업 충돌: 각자 다른 키 · 설정을 코드에 넣으면 매번 충돌

해결책은 값을 코드 밖으로 빼는 것입니다. 그런데 어디로 뺄까요? 여기서 두 종류로 나뉩니다.

2. 두 종류 — 비밀 vs 설정

| 구분 | <code>.env</code> (비밀) | <code>config.yaml</code> (설정) |
|--------|------------------------------------|-------------------------------|
| 담는 것 | API 키, 비밀번호 | 모델명·temperature·top_k·로그레벨 |
| Git 커밋 | ❌ 절대 금지(<code>.gitignore</code>) | ✅ 커밋(공개 가능) |
| 바뀌는 빈도 | 거의 안 바뀜 | 자주 튜닝 |
| 관리 주체 | 보안팀/개인(비공개) | 운영자(자유롭게 변경) |

핵심 기준:

비밀(secret)인가? → YES: `.env` (남이 보면 안 되는 것)
NO: `config.yaml` (남이 봐도 되는 설정)

API 키는 남이 보면 안 되니 `.env`. 모델명·온도는 공개돼도 되고 자주 바꾸니 `config.yaml`. 관리 주체가 다릅니다 — 키는 보안팀이 비공개로, 설정은 운영자가 자유롭게 바꾸죠.

3. `.env` — 비밀 보관

비밀은 `.env` 파일에 두고, `.gitignore`로 Git에서 제외합니다.

```
# .env (Git에 절대 올리지 않음)
GOOGLE_API_KEY=AIzaSy...본인키...
OPENAI_API_KEY=sk-...
```

코드에서는 `common.py`가 이미 이 `.env`를 읽어 줍니다(1강에서 설정).

```
from common import get_chat      # 내부에서 .env의 GOOGLE_API_KEY를 자동 로드
llm = get_chat()                 # 키를 코드에 안 적어도 됨
```


4. `config.yaml` — 설정 보관

행동을 바꾸는 값은 `config.yaml` 에 둡니다. 우리 프로젝트의 `data/config.yaml`:

```
# data/config.yaml (Git에 올려도 됨)
app:
  name: "승승장구 CS Agent"
  version: "1.0.0"
  language: "ko"

llm:
  provider: "gemini"          # gemini | openai
  model: "gemini-2.5-flash"
  temperature: 0.2
  max_retries: 3

rag:
  embed_model: "models/gemini-embedding-001"
  chunk_size: 500
  chunk_overlap: 50
  top_k: 4

logging:
  level: "INFO"
  file: "logs/agent.log"
```

여기 있는 값들은 코드 수정 없이 바꿀 수 있습니다. "온도를 0으로", "재시도를 5회로", "로그를 DEBUG로" — YAML만 고치면 되죠.

5. config 로드 코드

`pyyaml`로 YAML을 파이썬 dict로 읽습니다.

```
import yaml
from common import DATA

def load_config() -> dict:
    """data/config.yaml 을 읽어 dict로 반환한다(하드코딩 금지)."""
    with open(DATA / "config.yaml", encoding="utf-8") as f:
        return yaml.safe_load(f)          # YAML → 파이썬 dict
```

```

cfg = load_config()
llm = get_chat(provider=cfg["llm"]["provider"],      # 코드에 모델 · 온도를 안 박음
               temperature=cfg["llm"]["temperature"])
print(cfg["app"]["name"])                          # "승승장구 CS Agent"

```

- `yaml.safe_load` — YAML을 안전하게 dict로 변환(`load`가 아니라 `safe_load` — 보안).
- `cfg["llm"]["temperature"]` — 중첩된 설정을 dict 접근으로 꺼냄.
- LLM 생성 시 **config 값**을 넘김 → 코드엔 모델명·온도가 없음(하드코딩 아님).

6. 환경별 전환의 자유

설정을 분리하면 환경 전환이 코드 수정 없이 됩니다.

```

[개발 환경] config.yaml: temperature 0.5, logging DEBUG
[운영 환경] config.yaml: temperature 0.0, logging INFO
→ 같은 코드, config 파일만 다름

```

실무에선 `config.dev.yaml`, `config.prod.yaml`을 두고 환경변수로 어느 걸 읽을지 정하기도 합니다. 핵심은 "코드는 그대로, 설정만 바꿔" 환경을 전환한다는 것입니다.

핵심 정리

- 운영의 첫걸음은 **하드코딩 제거** — 키·설정을 코드 밖으로.
- 두 종류로 분리: 비밀(키)은 `.env`(Git 금지), 설정(모델·온도)은 `config.yaml`(Git OK).
- 기준: "남이 보면 안 되나?" → `.env`, 아니면 `config.yaml`.
- `yaml.safe_load`로 `config`를 dict로 읽어, LLM 생성에 **config 값**을 넘김.
- 설정 분리 덕에 **코드 수정 없이 환경 전환**(dev/prod).

구조적 로깅 — print를 버려라

운영에서 `print`는 못 씁니다. 레벨도, 시각도, 출처도 없으니까요. 표준 **logging** 모듈로 구조적 로깅을 합니다 — 시각·레벨(INFO/WARNING/ERROR)이 일관되게 붙고, 콘솔과 파일에 동시에 남아 운영 중 문제를 추적할 수 있습니다.

1. print의 한계

데모에선 `print` 로 충분했지만, 운영에선 무력합니다.

```
print("LLM 호출")          # 언제? 어느 수준의 일? 어디서?  
print("실패함")            # 얼마나 심각? 무시해도 되나?
```

문제점:

- 시각 없음: "언제" 일어난 일인지 모름
- 레벨 없음: 정보인지 경고인지 치명적 오류인지 구분 안 됨
- 출처 없음: 어느 모듈에서 찍혔는지 모름
- 켜고 끄기 불가: 운영에선 조용히, 디버깅 땀 상세히 → `print`는 못 함
- 파일 저장 안 됨: 터미널 닫으면 사라짐 → 나중에 못 봄

2. logging — 운영의 표준

파이썬 표준 `logging` 모듈은 이 모든 걸 해결합니다.

```
import logging  
logger = logging.getLogger("cs_agent")  
logger.info("LLM 호출 성공")          # 2026-06-01 10:00:01 [INFO] LLM 호출 성공  
logger.warning("재시도합니다")        # 2026-06-01 10:00:02 [WARNING] 재시도합니다  
logger.error("최종 실패")              # 2026-06-01 10:00:05 [ERROR] 최종 실패
```

시각 + 레벨 + 메시지가 자동으로 붙습니다. 나중에 로그 파일을 열어 "WARNING 이상만" 검색하거나, 특정 시각의 사건을 추적할 수 있죠.

3. 로그 레벨 — 심각도 구분

`logging`은 메시지에 심각도 레벨을 붙입니다.

```
DEBUG < INFO < WARNING < ERROR < CRITICAL  
(상세)                      (치명적)
```

```
DEBUG    : 개발 중 상세 추적 ("변수 x=5")
INFO     : 정상 동작 기록 ("앱 시작", "호출 성공")
WARNING  : 문제지만 계속 가능 ("재시도합니다")
ERROR    : 실패 ("호출 최종 실패")
CRITICAL : 시스템 다운 수준
```

레벨로 켜고 끕니다. `level=INFO`로 두면 DEBUG는 안 보이고 INFO 이상만 출력됩니다. 평소엔 INFO로 조용히, 문제 추적 땐 **config만 바꿔 DEBUG로** — 코드 수정 없이요.

```
# config.yaml 의 logging.level 만 "DEBUG"로 바꾸면 상세 로그가 켜짐
level = getattr(logging, cfg["logging"]["level"].upper(), logging.INFO)
logger.setLevel(level)
```

4. 핸들러 — 콘솔과 파일에 동시에

`logging`의 강력한 점은 여러 출력지(핸들러)에 동시에 보낼 수 있다는 것입니다.

하나의 `logger.info()` 호출이 두 곳으로 동시에 나갑니다.

- **StreamHandler** → 콘솔(터미널)
- **FileHandler** → `logs/agent.log` (파일에 영구 저장)

```
def setup_logging(cfg: dict) -> logging.Logger:
    level = getattr(logging, cfg["logging"]["level"].upper(), logging.INFO)
    log_path = ROOT / cfg["logging"]["file"]
    log_path.parent.mkdir(parents=True, exist_ok=True) # logs/ 자동 생성

    logger = logging.getLogger("cs_agent")
    logger.setLevel(level)
    logger.handlers.clear() # 재호출 시 핸들러 중복(로그 2번 찍힘) 방지
    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s") # 공통 포맷

    sh = logging.StreamHandler() # ① 콘솔 출력
    sh.setFormatter(fmt)
    fh = logging.FileHandler(log_path, encoding="utf-8") # ② 파일 출력
    fh.setFormatter(fmt)
    logger.addHandler(sh)
    logger.addHandler(fh)
    return logger
```

핵심 디테일:

- `StreamHandler` (콘솔) + `FileHandler` (파일) 둘 다 등록 → 화면으로 보면서 파일에도 남김.
- `Formatter` — `%(asctime)s [%(levelname)s] %(message)s` 로 시각·레벨·메시지 포맷 통일.
- `logger.handlers.clear()` — 이 함수를 여러 번 부르면 핸들러가 쌓여 로그가 2번, 3번 찍힙니다. 그래서 매번 비우고 다시 등록(흔한 함정).
- `mkdir(parents=True, exist_ok=True)` — `logs/` 폴더가 없으면 자동 생성.

5. 왜 파일에도 남기나

콘솔만으로는 부족합니다.

콘솔만: 터미널 닫으면 사라짐, 며칠 전 사건은 못 봄
파일에도: `logs/agent.log` 에 계속 쌓임
→ "어제 오후 3시에 무슨 일?" 을 파일 열어 검색 가능
→ 에러 빈도 집계, 사용자 문의 대응의 근거

운영에서 "왜 안 됐어요?"라는 문의가 오면, 로그 파일이 유일한 단서입니다. 파일에 남겨두지 않으면 영영 알 수 없죠. 그래서 운영 로그는 반드시 파일에도 남깁니다.

6. 사용 예

```
logger = setup_logging(load_config())
logger.info("앱 시작")
logger.info("LLM 호출 시도 1/3")
logger.warning("호출 실패(1회) → 2초 후 재시도")
logger.info("LLM 호출 성공")
```

출력 (콘솔 + `logs/agent.log` 동시)

```
2026-06-01 10:00:00 [INFO] 앱 시작
2026-06-01 10:00:00 [INFO] LLM 호출 시도 1/3
2026-06-01 10:00:01 [WARNING] 호출 실패(1회) → 2초 후 재시도
2026-06-01 10:00:03 [INFO] LLM 호출 성공
```

각 줄에 시각·레벨이 일관되게 붙어, 나중에 추적·검색·집계가 가능합니다.

핵심 정리

- 운영에선 print 를 버리고 **logging** 사용 — 시각·레벨·출처가 일관되게 붙음.
- **로그 레벨**(DEBUG~CRITICAL)로 심각도 구분, **config** 로 **레벨 전환**(코드 수정 없이).
- **핸들러 2 개**: StreamHandler(콘솔) + FileHandler(파일) → 화면+파일 동시.
- logger.handlers.clear()로 **중복 등록(로그 2 번)** 방지 — 흔한 함정.
- 파일 로그는 운영 문제 추적의 **유일한 단서** — 반드시 남긴다.

예외 처리와 재시도 (지수 백오프)

LLM/네트워크 호출은 **간헐적으로 실패**합니다(타임아웃·레이트리밋·일시 오류). 한 번 실패했다고 서비스가 죽으면 안 됩니다. **재시도(지수 백오프)** 로 일시 오류를 넘기고, 그래도 안 되면 **폴백 메시지**로 사용자에게 안내합니다. 서비스는 **절대 죽지 않게**.

1. LLM은 자주 흔들린다

LLM API 호출은 외부 네트워크를 타기 때문에 **간헐적으로 실패**합니다.

- 타임아웃: 응답이 너무 느려서 끊김
- 레이트리밋: 짧은 시간에 너무 많이 호출 (429 에러)
- 일시 오류: 서버 일시 과부하 (500, 503)
- 네트워크: 순간적인 연결 끊김

중요한 점: 이런 실패는 대부분 일시적입니다. 잠깐 기다렸다 다시 부르면 대개 성공하죠. 그러니 한 번 실패했다고 포기하면 안 됩니다.

2. 세 가지 전략 — 재시도·폴백·로깅

| 전략 | 설명 |
|------|---|
| 타임아웃 | 무한 대기 방지(요청 단위 시간 제한) |
| 재시도 | 일시적 실패는 지수 백오프로 N회 재시도 |
| 폴백 | 끝내 실패하면 사용자에게 안내 메시지 반환 (서비스는 죽지 않음) |
| 로깅 | 실패/재시도를 WARNING/ERROR로 기록 |

핵심 흐름:

```
호출 시도 → 실패? → 잠깐 기다림 → 재시도 → ... (N회까지)
                → N회 다 실패 → 폴백 메시지
```

3. 지수 백오프 — 기다리는 간격을 늘린다

재시도할 때 얼마나 기다리느냐가 중요합니다. 지수 백오프(**exponential backoff**) 는 간격을 점점 늘립니다.

```
1회 실패 → 1초 기다림 → 재시도
2회 실패 → 2초 기다림 → 재시도
3회 실패 → 4초 기다림 → 재시도
(1 → 2 → 4 → 8 ... 2배씩)
```

왜 간격을 늘리냐?

```
[고정 간격 (나쁨)] 실패 → 즉시 또 호출 → 또 실패 → 즉시 또...
                  → 과부하 상황에 계속 때려 서버 회복을 방해 (악화)

[지수 백오프 (좋음)] 실패 → 점점 더 기다림
                  → 과부하 서버에 '회복할 틈'을 줌 → 다음 시도 성공률 ↑
```

서버가 과부하라 실패하는 건데, 바로 또 때리면 더 망가집니다. 간격을 늘려 숨 쉴 틈을 주는 거죠.

4. 견고한 호출 래퍼

재시도 + 폴백을 하나의 함수로 묶습니다.

```
import time

def robust_invoke(llm, message: str, logger, max_retries: int = 3) -> str:
    """LLM 호출을 재시도+폴백으로 감싼다. 끝내 실패해도 서비스는 죽지 않는다."""
```

```

for attempt in range(1, max_retries + 1):
    try:
        logger.info(f"LLM 호출 시도 {attempt}/{max_retries}")
        resp = llm.invoke(message)
        logger.info("LLM 호출 성공")
        return resp.content                    # 성공하면 즉시 반환
    except Exception as e:                    # 타임아웃 · 레이트리미트 · 일시오류
        wait = 2 ** (attempt - 1)            # 지수 백오프: 1, 2, 4초
        logger.warning(f"LLM 호출 실패({attempt}회): {e} → {wait}초 후 재시도")
        time.sleep(wait)
# 끝까지 실패해도 예외로 죽지 않고, 사용자에게 안내 문구를 반환
logger.error("LLM 호출 최종 실패 — 폴백 응답 반환")
return "죄송합니다. 일시적인 오류로 답변을 생성하지 못했습니다. 잠시 후 다시 시도해 주세요."

```

한 줄씩 보기

- `for attempt in range(1, max_retries + 1)` — 최대 `max_retries` 회 시도(1, 2, 3).
- `try`: 성공하면 `return` — 더 재시도 안 함.
- `except Exception as e`: 모든 예외를 잡아 서비스가 안 죽게.
- `wait = 2 ** (attempt - 1)` — $2^0=1$, $2^1=2$, $2^2=4$ 초. 지수 백오프.
- `time.sleep(wait)` — 기다린 뒤 다음 루프(재시도).
- 루프를 다 돌고도 실패하면 → 폴백 메시지 반환(예외를 밖으로 안 던짐).

5. 폴백 — "안 죽는다"가 핵심

가장 중요한 부분은 마지막 줄입니다.

```
return "죄송합니다. 일시적인 오류로 답변을 생성하지 못했습니다. 잠시 후 다시 시도해 주세요."
```

[폴백 없음] 3번 실패 → 예외가 위로 던져짐 → 서버 500 에러 → 사용자: 빈 화면
 [폴백 있음] 3번 실패 → 안내 메시지 반환 → 사용자: "잠시 후 다시" (적어도 뭔가 받음)

완벽한 답을 못 줘도 사용자에게 정중한 안내라도 주는 것. 그게 "서비스가 안 죽는다"의 의미입니다. 24강 폴백("불완전해도 결과"), 26강 부분 실패 허용과 같은 정신이죠.

6. 실무 보강

- tenacity 라이브러리: @retry 데코레이터로 재시도를 선언적으로 작성
- ChatGoogleGenerativeAI(max_retries=N): Gemini 모델 자체의 재시도 옵션
- 지터(jitter): 백오프에 약간의 무작위를 더해 여러 클라이언트가 동시에 재시도하는 '썰림'을 분산

우리의 `robust_invoke` 는 원리를 보여주면서 풀백까지 책임지는 안전망입니다. 실무에선 `tenacity` 같은 라이브러리로 더 간결하게 쓰기도 하지만, 작동 원리는 똑같습니다.

핵심 정리

- LLM 호출은 일시적으로 자주 실패(타임아웃·레이트리밋) → 재시도하면 대개 성공.
- 지수 백오프: 재시도 간격을 1→2→4 초로 늘려 과부하 서버에 회복할 틈을 줌.
- robust_invoke: try/except + 재시도 루프 + 풀백 메시지.
- 핵심은 풀백 — 끝내 실패해도 예외로 죽지 말고 안내 메시지 반환(서비스 생존).
- 실패·재시도는 WARNING/ERROR 로깅으로 기록(추적).

비용·지연 관리

LLM 서비스는 호출할 때마다 돈과 시간이 듭니다. 토큰이 곧 비용이고, 프롬프트가 길수록 느립니다. 모델 선택·프롬프트 길이·캐싱·top_k 조절로 비용과 지연을 관리합니다. 운영의 마지막 4 번째 요소입니다.

1. 왜 비용·지연을 신경 써야 하나

데모는 몇 번 돌리고 말지만, 운영은 수천·수만 번 호출됩니다.

- 비용: 호출 1건이 0.002달러라도 × 10만 건 = 200달러/일
무료티어는 분당/일당 호출 한도가 있어 금방 막힘
- 지연: 응답이 5초 걸리면 사용자는 떠남. 빠를수록 좋은 경험

특히 토큰이 곧 비용입니다. 입력(프롬프트) + 출력(답변)의 토큰 수에 비례해 과금되죠(2강 토큰·비용 감각). 그래서 "꼭 필요한 만큼만" 쓰는 게 핵심입니다.

2. 전략 ① — 모델 선택

작업 난이도에 맞는 모델을 고릅니다. 모든 일에 최고 모델을 쓸 필요는 없습니다.

간단한 작업(분류 · 추출): gemini-2.5-flash-lite (싸고 빠름)
일반 작업(상담 · 요약): gemini-2.5-flash (균형)
어려운 작업(복잡 추론): 상위 모델만 골라서

```
# 작업에 따라 다른 모델 (config로 관리)
classify_llm = get_chat(...) # 분류는 가벼운 모델로 충분
answer_llm = get_chat(...) # 답변 생성만 좋은 모델
```

분류·라우팅 같은 단순 작업에 비싼 모델은 낭비입니다. 25강에서 규칙 라우터로 LLM 호출을 줄인 것도 같은 맥락 — 싸게 할 수 있는 건 싸게.

3. 전략 ② — 프롬프트 길이

토큰 = 비용이니, 프롬프트를 짧게 합니다.

[긴 프롬프트 (비쌈)]

"당신은 매우 친절하고 전문적이며 ... (강황한 역할 설명 10줄)
다음 규칙을 반드시 지켜주세요 ... (규칙 20개)"
→ 매 호출마다 이 토큰을 다 지불

[간결한 프롬프트 (저렴)]

"너는 승승장구물 상담원이다. 정책 문서 근거로만 답하라."
→ 핵심만, 토큰 절약

- 시스템 프롬프트를 핵심만 간결하게.
- RAG 컨텍스트를 너무 많이 넣지 않기(다음 전략).
- 대화 기록이 길어지면 트리밍/요약(19강).

4. 전략 ③ — top_k 조절 (RAG)

RAG에서 검색해 가져오는 청크 수(top_k)가 비용에 직결됩니다.

top_k=10: 관련 청크 10개를 다 프롬프트에 넣음 → 토큰 ↑↑ → 비쌈 · 느림
top_k=4: 상위 4개만 → 토큰 절약 (대개 충분)

```
# config.yaml
rag:
  top_k: 4          # 너무 많이 가져오지 않기
```

검색 결과를 많이 넣는다고 답이 좋아지지 않습니다. 오히려 관련 없는 청크가 노이즈가 되고 비용만 늘죠. "꼭 필요한 만큼만" 가져오는 게 핵심입니다(16강 검색 품질).

5. 전략 ④ — 캐싱

같은 질문이 반복되면, 매번 LLM을 부르지 말고 저장된 답을 돌려줍니다.

```
"무료배송 기준?" (첫 질문)  → LLM 호출 → 답 저장
"무료배송 기준?" (또 물음)  → 저장된 답 반환 (LLM 호출 0원 · 즉시)
```

```
# 개념: 질문을 key로 답을 캐시
cache = {}
def cached_answer(q):
    if q in cache:
        return cache[q]      # 캐시 적중 → LLM 안 부름
    ans = llm.invoke(q).content
    cache[q] = ans           # 결과 저장
    return ans
```

FAQ처럼 반복되는 질문이 많은 서비스에선 캐싱만으로 비용이 크게 줄어듭니다. 이건 워낙 효과가 커서, **08번에서 제대로 다룹니다**(메모리 + 파일 영속 캐시).

6. 운영 체크리스트

비용·지연을 포함한 29강 전체 운영 점검표입니다.

```
[ ] 키가 코드/Git에 없는가?      (.env + .gitignore)
[ ] 설정이 config.yaml로 외부화됐나?
[ ] 로그가 파일에도 남는가? 레벨 조절 가능?
[ ] LLM 호출에 타임아웃 · 재시도 · 폴백이 있나?
[ ] 실패 시 서비스가 죽지 않고 안내를 주나?
[ ] 비용 상한/모델 선택을 의식하고 있나?
[ ] 반복 질문에 캐싱을 적용했나?
[ ] RAG top_k가 과하지 않은가?
```

이 체크리스트를 통과하면 "내 PC에선 됐는데요"를 졸업한 것입니다.

핵심 정리

- LLM 서비스는 **호출마다 비용·시간** — 토큰이 곧 돈, 프롬프트가 길면 느림.
- ① **모델 선택**: 작업 난이도에 맞게(단순 작업에 비싼 모델 금지).
- ② **프롬프트 길이**: 시스템 프롬프트·컨텍스트를 간결하게.
- ③ **top_k 조절**: RAG는 꼭 필요한 청크만(과하면 노이즈+비용).
- ④ **캐싱**: 반복 질문은 저장된 답으로(08번에서 상세).
- **운영 체크리스트**를 통과하면 데모를 졸업한 운영 서비스.

따라하기 — 견고한 운영 래퍼

목표

02~05번에서 배운 설정 분리·로깅·재시도/폴백을 하나의 파일로 묶어, 운영 가능한 견고한 래퍼를 만듭니다. 실제 파일은 `code/ch29_deploy.py` 입니다.

0. 실습 준비

```
conda activate agentic

# 29강에서 쓰는 라이브러리
# - pyyaml : config.yaml 로드
# - langchain-google-genai : get_chat 내부 Gemini 모델
pip install -U pyyaml langchain-google-genai

python code/common.py # GOOGLE_API_KEY: True 확인
```

- 설정 파일 `data/config.yaml`은 이미 있습니다(app/llm/rag/logging).

1. 시작부 & 설정 로더

`code/ch29_deploy.py` 를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch29_deploy.py
import sys, pathlib, time, logging
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import DATA, ROOT, get_chat

import yaml

# [이 강의 핵심] '내 PC에선 됐는데요' 방지. 설정 분리 / 구조적 로깅 / 재시도+폴백으로
# 어디서 돌려도 똑같이 동작하고, 일시 오류에도 죽지 않는 견고한 서비스로 만든다.

def load_config() -> dict:
    """data/config.yaml 을 읽어 dict로 반환한다(하드코딩 금지)."""
    with open(DATA / "config.yaml", encoding="utf-8") as f:
        return yaml.safe_load(f)
```

`load_config` 는 02번에서 본 설정 로더입니다. 모델명·온도·재시도 횟수를 코드가 아니라 **config**에서 읽습니다.

2. 로깅 설정

03번의 구조적 로깅을 함수로 만듭니다.

```
def setup_logging(cfg: dict) -> logging.Logger:
    """config의 logging 설정으로 로거를 구성한다(콘솔+파일 동시 출력)."""
    level = getattr(logging, cfg["logging"]["level"].upper(), logging.INFO)
    log_path = ROOT / cfg["logging"]["file"]
    log_path.parent.mkdir(parents=True, exist_ok=True) # logs/ 자동 생성

    logger = logging.getLogger("cs_agent")
    logger.setLevel(level)
    logger.handlers.clear() # 재호출 시 핸들러 중복 방지
    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s")
```

```

sh = logging.StreamHandler(); sh.setFormatter(fmt) # 콘솔
fh = logging.FileHandler(log_path, encoding="utf-8"); fh.setFormatter(fmt) # 파일
logger.addHandler(sh); logger.addHandler(fh)
return logger

```

3. 견고한 LLM 호출 래퍼

04번의 재시도 + 폴백을 함수로 만듭니다.

```

def robust_invoke(llm, message: str, logger, max_retries: int = 3) -> str:
    """LLM 호출을 재시도+폴백으로 감싼다. 끝내 실패해도 서비스는 죽지 않는다."""
    for attempt in range(1, max_retries + 1):
        try:
            logger.info(f"LLM 호출 시도 {attempt}/{max_retries}")
            resp = llm.invoke(message)
            logger.info("LLM 호출 성공")
            return resp.content # 성공하면 즉시 반환
        except Exception as e:
            wait = 2 ** (attempt - 1) # 지수 백오프: 1, 2, 4초
            logger.warning(f"LLM 호출 실패({attempt}회): {e} → {wait}초 후 재시도")
            time.sleep(wait)
    logger.error("LLM 호출 최종 실패 — 폴백 응답 반환")
    return "죄송합니다. 일시적인 오류로 답변을 생성하지 못했습니다. 잠시 후 다시 시도해 주세요."

```

4. 조립 & 실행

세 함수를 묶어 main에서 흐름을 만듭니다.

```

def main():
    cfg = load_config() # 1) 설정 로드
    logger = setup_logging(cfg) # 2) 로깅 구성
    logger.info(f"앱 시작: {cfg['app']['name']} v{cfg['app']['version']}")

    # config의 값으로 LLM 생성(코드에 모델명 · 온도를 박지 않음 = 하드코딩 아님)
    llm = get_chat(provider=cfg["llm"]["provider"],
                    temperature=cfg["llm"]["temperature"])

```

```

answer = robust_invoke(
    llm, "승승장구물 무료배송 기준을 한 문장으로 알려줘.",
    logger, max_retries=config["llm"]["max_retries"],
)
print("\n[답변]", answer)
logger.info("앱 종료")

if __name__ == "__main__":
    main()

```

세 단계가 깔끔히 흐릅니다.

- 1) load_config() → 설정을 코드 밖에서 로드
- 2) setup_logging() → 콘솔+파일 로깅 준비
- 3) robust_invoke() → 재시도+폴백으로 안전하게 LLM 호출

5. 실행 결과

```

2026-06-01 10:00:00 [INFO] 앱 시작: 승승장구 CS Agent v1.0.0
2026-06-01 10:00:00 [INFO] LLM 호출 시도 1/3
2026-06-01 10:00:01 [INFO] LLM 호출 성공
[답변] 3만원 이상 구매 시 무료배송입니다.
2026-06-01 10:00:01 [INFO] 앱 종료

```

같은 로그가 `logs/agent.log` 파일에도 쌓입니다(03번 FileHandler).

관찰 포인트

- 키·모델·온도가 코드에 없습니다 — `.env` 와 `config.yaml` 에서 옴(하드코딩 제거).
- 모든 단계가 시각·레벨과 함께 로깅 — 나중에 추적 가능.
- LLM이 실패해도 재시도 → 폴백으로 서비스가 안 죽음.

6. 이게 "운영 코드"다

01번의 데모 코드와 비교해 봅시다.

| [데모 (01번)] | | [운영 (이 절)] |
|--------------|---|------------------------------|
| 키가 코드에 박힘 | → | .env / config.yaml 에서 로드 |
| 실패하면 죽음 | → | 재시도 + 폴백 (안 죽음) |
| print, 기록 없음 | → | logging 콘솔 + 파일 |
| 모델 · 온도 박제 | → | config.yaml 에서 (코드 수정 없이 변경) |

같은 "LLM에게 무료배송 기준 묻기"인데, 운영 품질이 완전히 다릅니다. 이 `ch29_deploy.py`의 세 함수(`load_config` · `setup_logging` · `robust_invoke`)는 재사용 가능한 운영 자산입니다. 07번 FastAPI, 08번 캐싱이 이 자산을 그대로 가져다 씁니다.

핵심 정리

- 02~05 번의 이론을 하나의 파일로 묶음: `load_config`·`setup_logging`·`robust_invoke`.
- main 흐름: 설정 로드 → 로깅 구성 → 견고한 호출.
- 키·모델·온도가 코드에 없고, 실패해도 안 죽고, 모든 게 로깅됨 = 운영 코드.
- 이 세 함수는 재사용 가능한 운영 자산 — 07·08 번이 그대로 활용.

따라하기 — FastAPI 엔드포인트로 감싸기

목표

06 번의 견고한 래퍼(`robust_invoke`)를 HTTP API 로 노출합니다.

CLI 데모로 끝내지 않고 POST /chat 엔드포인트로 만들면, 웹·앱 등 어떤 클라이언트도 우리 에이전트를 호출할 수 있습니다(운영 계약 구현).

0. 실습 준비

```
conda activate agentic

# FastAPI 서버 실행에 필요한 라이브러리
# - fastapi : 웹 API 프레임워크
# - uvicorn : FastAPI 앱을 실제로 구동하는 ASGI 서버
pip install -U fastapi uvicorn
```


1. 왜 HTTP API로 감싸나

06번까지는 `python ch29_deploy.py` 로 터미널에서만 돌렸습니다. 하지만 실제 서비스는 웹/앱에서 호출해야 하죠.

[CLI 데모] 나만, 터미널에서 직접 실행
[HTTP API] POST /chat 으로 → 웹페이지 · 모바일앱 · 다른 서버 누구나 호출

에이전트를 **HTTP 엔드포인트**로 노출하면, 28강에서 정한 운영 계약(`POST /chat {thread_id, message}`)이 실제로 구현됩니다.

2. 운영 자산 재사용

핵심은 06번의 함수들을 그대로 가져다 쓰는 것입니다.

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat

# 29강 운영 자산을 그대로 재사용 (설정 분리 · 구조적 로깅 · 재시도 래퍼)
from ch29_deploy import load_config, setup_logging, robust_invoke
```

새로 만드는 건 웹 계층뿐입니다. 비즈니스 로직(설정·로깅·재시도)은 06번 것을 재사용하죠. 이게 "본체와 전송 계층 분리"의 실제 모습입니다.

3. 앱·로거·LLM은 시작 시 한 번만

중요한 성능 포인트입니다.

```
try:
    from fastapi import FastAPI
    from pydantic import BaseModel

    # [핵심] 설정 · 로거 · LLM은 서버 시작 시 '한 번만' 만든다
    #           (요청마다 재생성하면 느리고 낭비)
    cfg = load_config()
    logger = setup_logging(cfg)
```

```
llm = get_chat(provider=cfg["llm"]["provider"],
               temperature=cfg["llm"]["temperature"])
logger.info(f"FastAPI 앱 초기화: {cfg['app']['name']} v{cfg['app']['version']}")

app = FastAPI(title="승승장구물 CS Agent")
```

[나쁨] 요청마다 load_config() · get_chat() 호출 → 매번 파일 읽고 객체 생성 (느림)
 [좋음] 서버 시작 시 1회 생성 → 모든 요청이 공유 (빠름)

설정·로거·LLM은 서버가 켜질 때 딱 한 번 만들고, 모든 요청이 이를 공유합니다.

4. 요청/응답 스키마와 엔드포인트

pydantic 모델로 입력·출력을 정의하면 자동 검증·문서화됩니다.

```
class ChatIn(BaseModel):
    thread_id: str = "demo"
    message: str
    # 요청 본문 스키마 = 입력 자동 검증

class ChatOut(BaseModel):
    thread_id: str
    reply: str
    # 응답 스키마(문서화 · 검증용)

@app.post("/chat", response_model=ChatOut)
def chat(body: ChatIn):
    """[POST /chat] 사용자 메시지를 받아 견고한 래퍼로 답변을 생성한다."""
    logger.info(f"요청 수신(thread={body.thread_id}): {body.message}")
    reply = robust_invoke(llm, body.message, logger,
                          max_retries=cfg["llm"]["max_retries"]) # 재시도+폴백
    return {"thread_id": body.thread_id, "reply": reply}

@app.get("/health")
def health():
    """[GET /health] 헬스체크 — 로드밸런서/모니터가 살아있는지 확인."""
    return {"status": "ok", "app": cfg["app"]["name"], "version": cfg["app"]["version"]}

except ImportError:
    app = None # fastapi 미설치여도 import 에러로 죽지 않게
    print("[안내] fastapi/uvicorn 미설치 — 'pip install fastapi uvicorn' 후 사용하세요.")
```

핵심

- `ChatIn` (요청 스키마) — `message` 가 없으면 FastAPI가 자동으로 422 에러를 냄(입력 검증 공짜).
- `@app.post("/chat")` — 이 함수가 `POST /chat` 요청을 처리.
- `robust_invoke` 로 호출 — 06번 재시도/폴백 래퍼를 그대로 사용(웹에서도 안 죽음).
- `/health` — 서버가 살아있는지 확인하는 헬스체크(운영 모니터링 필수).
- `try/except ImportError` — fastapi 미설치 환경에서도 파일이 깨지지 않게.

5. 서버 실행과 호출

서버는 **uvicorn**으로 띄웁니다.

```
# code/ 폴더에 ch29_app.py 로 배치했다고 가정
uvicorn ch29_app:app --reload
```

다른 터미널에서 호출:

```
# POST /chat 호출
curl -X POST localhost:8000/chat -H "Content-Type: application/json" \
  -d '{"thread_id":"u1","message":"무료배송 기준 알려줘"}'

# 응답
{"thread_id":"u1","reply":"3만원 이상 구매 시 무료배송입니다."}

# 헬스체크
curl localhost:8000/health
{"status":"ok","app":"승승장구 CS Agent","version":"1.0.0"}
```

6. 핵심 교훈 — 비즈니스 로직과 전송 계층 분리

```
[비즈니스 로직]  robust_invoke (06번) — "무엇을 어떻게 답하나, 안전하게"
+
[전송 계층]      FastAPI chat() — "HTTP로 받아 HTTP로 돌려준다"
```

웹 함수 `chat()` 은 **얇습니다** — 요청을 받아 `robust_invoke` 를 부르고 결과를 반환할 뿐. 진짜 로직은 06번 래퍼에 있죠. 이렇게 분리하면:

- robust_invoke 는 CLI에서도, FastAPI에서도, 테스트에서도 재사용
- HTTP를 Slack 봇이나 gRPC로 바꿔도 비즈니스 로직은 그대로

핵심 정리

- 06 번 래퍼를 **FastAPI POST /chat** 으로 노출 → 웹·앱 등 누구나 호출 가능.
- 설정·로거·LLM 은 **서버 시작 시 1 회만** 생성(요청마다 재생성 금지).
- pydantic 스키마(ChatIn)로 **입력 자동 검증** + /docs 자동 문서.
- /health 헬스체크로 운영 모니터링 대비.
- 핵심 교훈: **비즈니스 로직(robust_invoke)과 전송 계층(HTTP)을 분리** → 재사용·교체 용이.

응답 캐싱으로 비용 줄이기

같은 질문이 반복되면 **LLM 을 다시 부르지 말고 저장된 답을 돌려줍니다**. 메모리 dict 캐시 + JSON 파일 영속 캐시를 만들어, 반복 질문을 **0 원·즉시** 처리합니다. 히트/미스 로그와 절감된 호출 수·예상 비용까지 출력합니다.

1. 왜 캐싱인가

FAQ형 서비스에선 같은 질문이 수없이 반복됩니다.

"무료배송 기준?" ← 하루에 수백 명이 똑같이 물음

"환불 며칠 걸려?" ← 역시 반복

[캐싱 없음] 매번 LLM 호출 → 매번 돈·시간 (같은 답을 매번 새로 생성)

[캐싱 있음] 첫 질문만 LLM → 이후엔 저장된 답 (0원·즉시)

LLM 호출은 돈과 시간이 듭니다. 같은 질문에 **매번 새로 묻는 건 낭비**죠. 질문을 key로 답을 저장해 두면, 반복 질문은 공짜로 처리됩니다(05번 전략 ④).

2. 캐시 key — 질문 정규화

문제는 "같은 질문"을 어떻게 판단하느냐입니다. 글자가 조금만 달라도 다른 질문으로 보면 캐시가 잘 안 맞죠.

```
import hashlib

def _key(question: str) -> str:
    """질문을 정규화 후 해시해 캐시 key로 쓴다(공백 · 대소문자 차이 흡수)."""
    norm = " ".join(question.strip().lower().split()) # 앞뒤 공백 · 중복 공백 · 대소문자 정규화
    return hashlib.sha256(norm.encode("utf-8")).hexdigest()[:16]
```

"무료배송 기준?", "무료배송 기준?", " 무료배송 기준? " — 셋 다 정규화하면 같은 key → 캐시 적중!

- `strip()` — 앞뒤 공백 제거.
- `lower()` — 대소문자 통일(영문).
- `" ".join(...split())` — 중복 공백을 하나로.
- `sha256(...)[:16]` — 정규화된 질문을 짧은 해시로(key로 쓰기 좋게).

이렇게 정규화하면 사소한 차이를 흡수해 캐시 적중률이 올라갑니다.

3. 캐시 클래스 — 메모리 + 파일 영속

메모리 dict로 빠르게, JSON 파일로 서버 재시작에도 유지되게 만듭니다.

```
import json
from common import ROOT

CACHE_FILE = ROOT / "cache" / "response_cache.json"

class ResponseCache:
    """질문→답변 캐시. 메모리 dict + JSON 파일 영속."""

    def __init__(self, persist: bool = True):
        self.persist = persist
        self.store = {} # key → answer
        self.hits = 0 # 캐시 히트 횟수(= 절감된 호출 수)
        self.misses = 0 # 캐시 미스 횟수(= 실제 LLM 호출)
        if persist and CACHE_FILE.exists():
            self.store = json.loads(CACHE_FILE.read_text(encoding="utf-8")) # 기존 캐시 로드
```

```
def get(self, question: str):
    return self.store.get(_key(question))

def set(self, question: str, answer: str):
    self.store[_key(question)] = answer
    if self.persist:
        # 파일에도 즉시 반영(서버 재시작에도 유지)
        CACHE_FILE.parent.mkdir(parents=True, exist_ok=True)
        CACHE_FILE.write_text(
            json.dumps(self.store, ensure_ascii=False, indent=2), encoding="utf-8")
```

메모리 dict: 실행 중 빠른 조회 (RAM)
 JSON 파일: 서버를 켜다 켜도 캐시 유지 (디스크 영속)
 → 시작 시 파일 로드, 저장 시 파일에도 기록

- `hits` / `misses` 카운터로 절감 효과를 추적.
- `ensure_ascii=False` — 한글이 `\uXXXX` 로 깨지지 않게 그대로 저장.

4. 캐시 우선 호출

캐시를 먼저 보고, 없을 때만 LLM(06번 `robust_invoke`)을 부릅니다.

```
from ch29_deploy import robust_invoke

def cached_invoke(cache: ResponseCache, llm, question: str, logger) -> str:
    """캐시를 먼저 보고, 없을 때만 robust_invoke로 LLM을 부른다."""
    hit = cache.get(question)
    if hit is not None:
        cache.hits += 1
        logger.info(f"캐시 HIT — LLM 호출 건너뛴: {question}")
        return hit
        # 캐시 적중 → LLM 안 부름
    cache.misses += 1
    logger.info(f"캐시 MISS — LLM 호출: {question}")
    answer = robust_invoke(llm, question, logger) # 견고한 래퍼로 실제 호출
    cache.set(question, answer) # 결과를 캐시에 저장
    return answer
```

질문 → 캐시 확인:

- HIT → 저장된 답 반환 (LLM 호출 0)
- MISS → `robust_invoke` 호출 → 답 저장 → 반환

06 번 운영 자산(robust_invoke)을 그대로 재사용합니다 — 캐시는 그 앞단에 얹은 층입니다.

5. 절감 효과 리포트

캐시가 얼마나 아꼈는지 보여줍니다.

```
COST_PER_CALL = 0.002  # 호출 1회당 예상 비용(USD, 데모용 가정치)

def report(self):
    total = self.hits + self.misses
    saved = self.hits * COST_PER_CALL
    print(f"[캐시 리포트] 총 요청 {total}건 / 히트 {self.hits} / 미스 {self.misses}")
    print(f" 절감된 LLM 호출: {self.hits}회 → 예상 절감 비용: ${saved:.4f}")
```

6. 실행과 결과

같은 질문을 일부러 반복해 캐시 효과를 확인합니다.

```
if __name__ == "__main__":
    cfg = load_config()
    logger = setup_logging(cfg)
    llm = get_chat(provider=cfg["llm"]["provider"], temperature=cfg["llm"]["temperature"])
    cache = ResponseCache(persist=True)

    questions = [
        "승승장구물 무료배송 기준을 한 문장으로 알려줘.",
        "승승장구물 무료배송 기준을 한 문장으로 알려줘.", # 동일 → HIT
        "환불은 며칠 안에 가능해?",
        "승승장구물 무료배송 기준을 한문장으로 알려줘.", # 공백만 다름 → 정규화로 HIT
        "환불은 며칠 안에 가능해?", # 동일 → HIT
    ]
    for i, q in enumerate(questions, 1):
        ans = cached_invoke(cache, llm, q, logger)
        print(f"[{i}] 답변: {ans[:60]}\n")
    cache.report()
```

예상 출력

[MISS] 승승장구물 무료배송 기준... → LLM 호출
[1] 답변: 3만원 이상 구매 시 무료배송입니다.

[HIT] 승승장구물 무료배송 기준...
[2] 답변: 3만원 이상 구매 시 무료배송입니다.

[MISS] 환불은 며칠 안에 가능해? → LLM 호출
[3] 답변: 환불은 영업일 기준 3~5일 내 처리됩니다.

[HIT] 승승장구물 무료배송 기준... ← 공백만 다른데도 정규화로 적중!
[4] ...

[캐시 리포트] 총 요청 5건 / 히트 3 / 미스 2
절감된 LLM 호출: 3회 → 예상 절감 비용: \$0.0060

관찰 포인트

- 5번 질문 중 실제 **LLM 호출은 2번뿐** — 3번은 캐시로 처리(60% 절감).
- 4번 질문은 공백만 달랐는데도 정규화 덕에 적중했습니다(2번 캐시 key 기법).
- `response_cache.json`에 저장돼, 서버를 꺾다 켜도 캐시가 유지됩니다.

7. 주의 — 캐싱이 부적절한 경우

캐싱 좋음: FAQ·정책 안내처럼 '답이 고정'된 질문
캐싱 주의: "내 주문 상태는?" → 시시각각 변함! 캐싱하면 옛 정보를 줌
→ 실시간 도구 호출 결과는 캐싱하면 안 됨 (또는 짧은 TTL)

답이 변하는 질문은 캐싱하면 안 됩니다. 주문 상태·재고처럼 실시간 데이터는 캐싱하면 옛 정보를 주게 되죠. FAQ·정책처럼 답이 고정된 것만 캐싱하는 게 안전합니다. (실무에선 캐시에 **유효기간 (TTL)**을 뒤 오래된 답을 자동 폐기합니다.)

핵심 정리

- 반복 질문은 저장된 답으로 처리 → LLM 호출 0 원·즉시(05 번 전략 ④의 구현).
- 캐시 key 정규화(공백·대소문자 흡수 + 해시)로 적중률 ↑.
- 메모리 dict + JSON 파일 영속 → 빠르고, 서버 재시작에도 유지.
- cached_invoke 가 06 번 robust_invoke 앞단에 얹혀 캐시 우선 처리.
- 히트/미스·절감 비용 리포트로 효과 측정.
- 답이 변하는 질문(주문·재고)은 캐싱 금지 — FAQ·정책만(또는 TTL).

실습문제

문제 1 — 재시도 + 캐싱 + 로깅 강화

요구사항

`code/ch29_deploy.py` 를 복사해 `code/ch29_ex_robust.py` 를 만드세요.

- `robust_invoke` 에 질문 단위 캐시(dict 또는 `functools.lru_cache`)를 붙여, 동일 질문 반복 시 LLM 을 호출하지 않고 캐시로 답한다.
- 각 호출마다 `logger` 로 "캐시 적중/미스", 시도 횟수, 소요시간을 기록한다.
- `max_retries` 는 `config.yaml` 의 `llm.max_retries` 에서 읽는다.
- 기대 결과: 같은 질문을 두 번 던지면 두 번째에 "캐시 적중" 로그가 찍히고 LLM 재호출이 없다.

문제 2 — FastAPI 서버 작성

요구사항

`code/ch29_ex_api.py` 를 작성하세요.

- FastAPI로 `POST /ask` (body: `{"question": "...}"`) 엔드포인트를 만든다.
- 내부에서 `robust_invoke` 로 LLM을 호출하고 `{"answer": "...}"` 를 반환한다.
- 예외 시 500 대신 `{"error": ...}` 를 반환한다.
- 검증: `uvicorn ch29_ex_api:app` 로 띄운 뒤 `curl -X POST localhost:8000/ask ... -d '{"question":"무료 배송 기준"}'` 로 정상 JSON 응답 확인.